CARTEIRA DIGITAL

Documentação da Arquitetura de Software — Entrega 2

Template arc42 versão 8.2-PT · C4 Model níveis 1 a 4 · UML · DER · Software Architecture Canvas

---

Estilos arquiteturais aplicados: Serverless · Microfrontend · Backend for Frontend (BFF) + Microsserviços + Database per Service · API Gateway · Arquitetura Orientada a Eventos (EDA) · Clean Architecture e Vertical Slice

---

INTEGRANTES DO GRUPO

Gabriel João Conti

Guilherme Celli Fadel

Lucas Arthur da Silva Reali

Thiago Akimoto

Arquitetura de Soluções com Cloud

Bacharelado em Engenharia de Software

Outubro de 2026

Sumário

1. Introdução e Objetivos 4
   1.4 Software Architecture Canvas 5
2. Restrições Arquiteturais 7
3. Contexto e Escopo 8
   3.1 Contexto Negocial (C4 Nível 1) 8
   3.2 Contexto Técnico 10
4. Estratégia de Solução 11
   4.2 Contêineres (C4 Nível 2) 12
5. Visão de Blocos de Construção 16
   5.2 Componentes (C4 Nível 3) 17
6. Visão de Tempo de Execução 20
   6.1 Sequência UML (C4 Nível 4) 20
   6.2 Componentes UML (C4 Nível 4) 22
   6.3 Classes UML (C4 Nível 4) 24
7. Visão de Implantação 27
8. Conceitos Transversais 30
   8.1 Modelo de Dados (DER) 30
9. Decisões Arquiteturais 34
10. Requisitos de Qualidade 37
11. Riscos e Débitos Técnicos 38
12. Glossário 39

Índice de Diagramas

Os diagramas seguem o mapeamento entre arc42 e C4 Model adotado na disciplina: diagrama de contexto na seção 3 (Contexto e Escopo), diagramas de contêineres na seção 4 (Estratégia de Solução), diagrama de componentes na seção 5 (Blocos de Construção) e diagramas de código, o nível 4, na seção 6, todos em UML (sequência, componentes e classes). O Software Architecture Canvas está na seção 1, o diagrama de implantação na seção 7 e o DER na seção 8. Todos foram feitos no draw.io; o arquivo-fonte `carteira-digital-diagramas.drawio` acompanha esta entrega.

Figura	Diagrama	Notação	Seção
1	Software Architecture Canvas	Canvas oficial do arc42	1.4
2	Diagrama de Contexto do Sistema (Nível 1)	C4 Model	3.1
3	Diagrama de Contêineres 1/3 – canais, API Gateway e BFFs (Nível 2)	C4 Model	4.2
4	Diagrama de Contêineres 2/3 – microsserviços, dados e integrações (Nível 2)	C4 Model	4.2
5	Diagrama de Contêineres 3/3 – eventos e serverless (Nível 2)	C4 Model	4.2
6	Diagrama de Componentes do Serviço de Carteira (Nível 3)	C4 Model	5.2
7	Diagrama de Sequência da transferência entre carteiras (Nível 4)	UML 2.5.1	6.1
8	Diagrama de Componentes da Fatia Transferir (Nível 4)	UML 2.5.1	6.2
9	Diagrama de Classes da Fatia Transferir e do domínio (Nível 4)	UML 2.5.1	6.3
10	Diagrama de Implantação na Microsoft Azure	C4 Model (implantação)	7.1
11	Diagrama de Entidades e Relacionamentos (um banco por serviço)	Entidade-Relacionamento, pé de galinha	8.1

Como ler esta entrega

O que mudou desde a primeira entrega. A primeira versão documentava um monolito modular. Esta entrega mantém o mesmo sistema e os mesmos requisitos (RF-001 a RF-075), mas reorganiza a arquitetura para aplicar os estilos exigidos: microsserviços com um banco por serviço, BFF por canal, API Gateway, microfrontends, eventos e funções serverless. A decisão antiga foi substituída e o motivo está na ADR-01 (seção 9).

Arquitetura-alvo × incremento implementado. Este documento descreve a arquitetura a ser implementada (arquitetura-alvo). O código desenvolvido até aqui é o primeiro incremento: os módulos `users` (futuro Serviço de Identidade) e `wallets` (futuro Serviço de Carteira), ambos em Clean Architecture com Vertical Slice. Neste incremento os dois módulos ainda rodam juntos, em um único executável Spring Boot com banco H2 em memória; a separação em dois serviços com PostgreSQL é o próximo passo (risco R-1). Todo o restante é projetado.

Como identificar cada coisa. O texto marca os itens com **implementado** ou **projetado**. Os diagramas de contexto, contêineres e implantação (Figuras 2 a 5 e 10) mostram a arquitetura-alvo. Os diagramas de componentes, de código e o DER (Figuras 6 a 9 e 11) refletem o código; neles, os elementos com borda tracejada ainda não foram implementados.

Onde cada item da atividade está

Item exigido	Onde está neste documento
arc42: template oficial, com as 11 seções exigidas e a seção 12 (Glossário)	Seções 1 a 12; diagramas nas respectivas seções (Índice de Diagramas)
Software Architecture Canvas	Seção 1.4, Figura 1
C4 Model Nível 1 – Contexto	Seção 3.1, Figura 2
C4 Model Nível 2 – Contêineres, com portas, endereços e protocolos	Seção 4.2, Figuras 3, 4 e 5
C4 Model Nível 3 – Componentes, com portas, endereços e protocolos	Seção 5.2, Figura 6
C4 Model Nível 4 – UML de Sequência, Componentes e Classes	Seção 6.1 (Figura 7), 6.2 (Figura 8) e 6.3 (Figura 9)
Diagrama de Entidades e Relacionamentos	Seção 8.1, Figura 11
Diagrama de implantação	Seção 7.1, Figura 10
Serverless	Funções de Notificação: seções 4.1 e 6.4, Figuras 5 e 10, ADR-06
Microfrontend	Shell + MFEs com Module Federation: seções 4.1, 6.5 e 7.2, Figura 3, ADR-04
BFF + Microsserviços + Database per Service	Seções 4.1, 5.1 e 8.1, Figuras 3, 4 e 11, ADR-01, ADR-03 e ADR-09
API Gateway	Azure API Management: seções 4.1 e 7.1, Figuras 3 e 10, ADR-03
Arquitetura Orientada a Eventos (EDA)	Barramento de Eventos: seções 4.1, 5.1, 6.4 e 8.5, Figura 5, ADR-05
Clean Architecture e Vertical Slice refletindo o código	Seções 5.2 e 6, Figuras 6 a 9, ADR-02 (nomes de pacotes, classes e métodos iguais aos do código)
Integrantes do grupo	Capa: Gabriel João Conti, Guilherme Celli Fadel, Lucas Arthur da Silva Reali e Thiago Akimoto

1. Introdução e Objetivos

A Carteira Digital é um aplicativo de conta digital. Ela permite que uma pessoa abra uma conta pelo celular, envie e receba dinheiro por PIX, pague contas, use cartões e acompanhe seu extrato. Uma equipe interna de backoffice aprova cadastros e atende chamados por um portal web.

O sistema existe para dar acesso a serviços financeiros básicos sem exigir ida a uma agência: todo o relacionamento acontece pelo celular, da abertura da conta ao suporte.

1.1 Visão Geral dos Requisitos

Os requisitos funcionais completos estão no documento *Requisitos_Funcionais_Carteira_Digital.xlsx* (RF-001 a RF-075), entregue na primeira atividade. A tabela mostra o resumo por módulo e qual contêiner da arquitetura-alvo responde por ele.

Módulo	Requisitos	O que faz	Responsável na arquitetura
Onboarding e KYC	RF-001 a RF-009	Cadastro, validação de documento e biometria, abertura da conta	Serviço de Clientes **projetado**
Autenticação e Segurança	RF-010 a RF-018	Login, biometria, MFA, PIN de transação, sessões	Serviço de Identidade **implementado** cadastro de credenciais e login com JWT
Conta e Saldo	RF-019 a RF-025	Saldos, registro de movimentações, bloqueios	Serviço de Carteira **implementado** abertura, depósito, saque, transferência e saldo
PIX	RF-026 a RF-035	Chaves, QR Code, envio, recebimento e devolução	Serviço de Pagamentos **projetado**
Transferências e Pagamentos	RF-036 a RF-042	TED, boletos, tributos, recargas, comprovantes	Serviço de Pagamentos **projetado**
Cartões	RF-043 a RF-049	Cartão virtual e físico, limites, fatura	Serviço de Cartões **projetado**
Extrato e Histórico	RF-050 a RF-054	Extrato com filtros, exportação, categorização	Serviço de Carteira **implementado** extrato do mais recente ao mais antigo
Notificações	RF-055 a RF-057	Push, e-mail e SMS	Funções de Notificação (serverless) **projetado**
Antifraude e Compliance	RF-058 a RF-064	Análise de risco, listas restritivas, LGPD	Serviço Antifraude **projetado**
Backoffice	RF-065 a RF-070	Consultas, aprovação de cadastro, chamados	Portal de Backoffice (microfrontends) + BFF Backoffice + Serviço de Clientes **projetado**
Integrações	RF-071 a RF-075	Banco Central, cartões, API pública	API Gateway + integração de cada serviço com seu parceiro **projetado**

Os casos de uso que mais influenciam a arquitetura são: abrir conta pelo celular (RF-001 a RF-008), enviar PIX (RF-027 e RF-029), transferir entre carteiras, consultar saldo e extrato (RF-019 e RF-050) e aprovar cadastro no backoffice (RF-066).

O incremento já implementado atende, de ponta a ponta e com testes, a este recorte do escopo: `POST /users` (cadastro), `POST /auth/login` (token JWT), `POST /wallets` (abrir carteira), `POST /wallets/{id}/deposits`, `POST /wallets/{id}/withdrawals`, `POST /wallets/{id}/transfers`, `GET /wallets/{id}/balance` e `GET /wallets/{id}/transactions`.

1.2 Objetivos de Qualidade

#	Objetivo	O que significa neste sistema	Cenário
1	Segurança	Ninguém movimenta dinheiro sem ser o dono da conta, e dados pessoais não vazam	Um usuário autenticado tenta sacar da carteira de outra pessoa: o sistema recusa (403) e o saldo não muda

#	Objetivo	O que significa neste sistema	Cenário
2	Integridade dos dados financeiros	O saldo sempre bate com a soma dos lançamentos; nenhuma transação some ou duplica	Duas transferências simultâneas da mesma carteira: uma é aplicada e a outra é recusada com 409, sem saldo negativo
3	Manutenibilidade	Cada serviço e cada fatia evolui, é testado e é implantado sem quebrar os outros	Um integrante cria uma nova fatia na Carteira e nenhum teste das fatias existentes quebra
4	Escalabilidade elástica	Picos de uso escalam só a parte afetada, e o que está ocioso não gera custo	Num pico de PIX, só o Serviço de Pagamentos e as Funções de Notificação ganham réplicas
5	Usabilidade	Uma pessoa sem treino consegue enviar um PIX na primeira tentativa	Alguém que nunca usou o aplicativo envia um PIX por chave, sem ajuda, em menos de 3 minutos

Segurança e integridade vêm primeiro porque o sistema movimenta dinheiro, e uma falha aqui é irreversível. Manutenibilidade e escalabilidade são as qualidades que justificam a troca do monolito por microsserviços: cada serviço pode ser evoluído por uma pessoa e escalado sozinho. Desempenho continua sendo acompanhado pelos cenários da seção 10.

1.3 Partes Interessadas

Papel	Quem é	Expectativa em relação à arquitetura
Cliente	Pessoa física que abre e usa a conta	Enviar dinheiro em poucos toques, sem travar e sem risco de perder valores
Operador de Backoffice	Funcionário que aprova cadastros e atende chamados	Um portal interno que mostre o histórico do cliente e permita agir sobre a conta
Equipe de desenvolvimento	Os quatro integrantes do grupo	Arquitetura dividida em frentes independentes (serviços e fatias), para trabalhar em paralelo sem conflito
Professor da disciplina	Avaliador da entrega	Documentação aderente ao arc42, com notação correta de C4 Model e UML e com os estilos arquiteturais exigidos aplicados
Parceiros externos	Banco Central, banco parceiro, bureau de biometria, processadora de cartões e serviço de mensagens	Integrações que respeitem o contrato de cada serviço, com autenticação forte e rastreabilidade
Encarregado de dados (LGPD)	Responsável pelo tratamento de dados pessoais	Dados pessoais minimizados, cifrados e com registro de acesso

1.4 Software Architecture Canvas

A Figura 1 é o **Software Architecture Canvas** da Carteira Digital, preenchido sobre o template oficial do arc42 (canvas.arc42.org, onde o mesmo canvas também é apresentado com o nome Architecture Inception Canvas). Títulos, textos-guia e o rodapé de licença (CC BY-SA 4.0) são os do template; só o conteúdo é nosso. O canvas resume em uma página o que orienta o restante do documento, nas três partes do modelo: **objetivos** (caso de negócio, visão funcional e as três metas de qualidade principais, que são as três primeiras da seção 1.2), **solução** (contexto de negócio, restrições organizacionais e técnicas e hipóteses arquiteturais) e **avaliação** (desafios técnicos e riscos).

Cada bloco do canvas corresponde a uma seção deste documento: visão funcional → 1.1; metas de qualidade → 1.2; contexto de negócio → 3.1; restrições organizacionais e técnicas → 2; hipóteses arquiteturais → 9 (ADRs); desafios técnicos e riscos → 11. Em relação à primeira entrega, as hipóteses passaram a refletir os estilos arquiteturais adotados e os riscos incluem os efeitos da distribuição em microsserviços.

1.4 Software Architecture Canvas

Software Architecture Canvas

System:	Created by:	Created for:	Date / Iteration:
Carteira Digital	G. J. Conti · G. C. Fadel · L. A. S. Reali · T. Akimoto	Arquitetura de Soluções com Cloud	Outubro de 2026 · iteração 2

Business Case

Brief description of the business case or economic driver behind the software system.

- Dar acesso a serviços financeiros básicos sem ida a uma agência: abertura de conta, PIX, pagamentos, cartões e extrato pelo celular, com portal interno para aprovação de cadastros e atendimento.

Functional Overview

The most important functional requirements at a high level.

- Identidade: login, MFA, PIN transacional e JWT (RF-010 a RF-018)
- Clientes: cadastro, KYC, aprovação e chamados (RF-001 a RF-009, RF-065 a RF-070)
- Carteira: conta, saldo, lançamentos e extrato (RF-019 a RF-025, RF-050 a RF-054)
- Pagamentos: PIX, TED, boletos, tributos e recargas (RF-026 a RF-042)
- Cartões: virtual e físico, limites e fatura (RF-043 a RF-049)
- Antifraude e compliance (RF-058 a RF-064)
- Notificações por push, e-mail e SMS (RF-055 a RF-057)
- Integrações e API pública (RF-071 a RF-075)

Quality Goals

The three most important quality goals for the architecture, which have the highest priority for the most important stakeholder.

- **1. Segurança**: ninguém movimenta dinheiro sem ser o dono da conta; dados pessoais não vazam
- **2. Integridade financeira**: o saldo sempre bate com os lançamentos; nada some nem duplica
- **3. Manutenibilidade**: cada serviço e cada fatia evolui e é implantado sem quebrar os outros

Business Context

Separate your system under construction as a black box from all its communication partners. Communication partners are neighbouring external systems and users.

- **Sistema em construção (caixa preta)**
  Carteira Digital: conta digital com PIX, pagamentos, cartões e extrato
- **Usuários**
  Cliente: abre a conta, transfere, paga e consulta saldo
  Operador de backoffice: aprova cadastros e atende chamados
- **Sistemas vizinhos**
  Banco Central (PIX): chaves no DICT e liquidação no SPI
  Banco parceiro: TED, boletos e tributos
  Validação e biometria: confere CPF e selfie
  Processadora de cartões: emissão e autorização de compras
  Serviço de mensagens: push, e-mail e SMS
- **Observação**
  Os parceiros externos são simulados neste projeto (RT-1 e RT-2), com a mesma interface dos serviços reais

Organisational Constraints

Any organisational requirement that limits the software architects freedom of decision.

- RO-1 Equipe fixa de 4 integrantes, estudantes em período noturno
- RO-2 Prazo do semestre letivo, com entregas parciais no AVA
- RO-3 Mesmo sistema em todas as atividades da disciplina
- RO-4 Uso obrigatório de arc42, C4 Model e UML e dos estilos Serverless, Microfrontend, BFF + Microsserviços + Database per Service, API Gateway, EDA, Clean Architecture e Vertical Slice

Technical Constraints

Any technical requirement that restricts the software architects freedom of decision

- RT-1 Sem acesso real ao SPI/DICT do Banco Central
- RT-2 Sem acesso a processadora, bureau de biometria e banco liquidante
- RT-3 Backend em Java 25 com Spring Boot 4 (stack do código entregue)
- RT-4 PostgreSQL, um banco por microsserviço
- RT-5 App em React Native; portal em React com Module Federation
- RT-6 Documentação arc42 com diagramas C4 e UML em draw.io
- RT-7 Microsoft Azure com orçamento acadêmico: planos de consumo e escala a zero

Architectural Hypotheses

Resulting architectural hypotheses and important, expensive, large-scale or risky decisions, including justifications.

- ADR-01 Microsserviços com um banco por serviço, no lugar do monolito modular da entrega 1
- ADR-02 Clean Architecture + Vertical Slice dentro de cada serviço (já no código)
- ADR-03 API Gateway único (Azure API Management) e um BFF por canal
- ADR-04 Microfrontends com Module Federation no portal de backoffice
- ADR-05 Eventos de negócio no Azure Service Bus, com outbox transacional
- ADR-06 Notificações em Azure Functions (serverless, disparadas por evento)
- ADR-07 Bloqueio otimista e gravação ordenada para o saldo
- ADR-08 Simuladores com a interface dos parceiros reais · ADR-09 Um servidor PostgreSQL, um banco por serviço

Technical Challenges & Risks

Identified current known challenges and technical risks

- R-1 O código cobre 2 dos 6 serviços e ainda sobe como um único processo com banco compartilhado
- R-2 Eventos e outbox ainda não implementados
- R-3 Consistência entre serviços passa a ser eventual: exige sagas, idempotência e outbox
- R-4 Segredo HS256 compartilhado para validar o JWT em vários serviços
- R-5 Mais peças para operar e observar com uma equipe de 4 pessoas
- R-6 Integração real com SPI/DICT nunca exercitada; simuladores não reproduzem falhas reais
- R-7 Partida a frio e custo da Azure se a escala a zero não for bem configurada

Software Architecture Canvas, © 2023 by Gernot Starke, Patrick Roos and arc42 Contributors is licensed under Attribution-ShareAlike 4.0 International.

https://canvas.arc42.org

Figura 1 — Software Architecture Canvas da Carteira Digital (template oficial arc42, canvas.arc42.org)

2. Restrições Arquiteturais

As restrições abaixo limitam a liberdade de decisão da equipe: não são escolhas de projeto, são condições dadas que a arquitetura precisa acomodar.

2.1 Restrições Técnicas

#	Restrição	Motivo	Consequência
RT-1	Sem acesso real ao SPI/DICT do Banco Central	Exige ser instituição autorizada	O PIX é integrado contra um simulador com a mesma interface do serviço real
RT-2	Sem acesso a processadora de cartões, bureau de biometria e banco liquidante	Contratos comerciais	Todos os parceiros externos são simulados
RT-3	Backend em Java 25 com Spring Boot 4.1 e Gradle 9	Stack conhecida pela equipe e usada no código do incremento	Todos os microsserviços e BFFs usam o mesmo stack, o que facilita reaproveitar o padrão de fatias
RT-4	Banco relacional PostgreSQL, um banco por microsserviço	Dados financeiros exigem transação ACID; Database per Service exige isolamento	Cada serviço tem banco e usuário próprios. Em desenvolvimento, testes e no incremento atual o código usa H2 em memória
RT-5	Aplicativo em React Native; portal de backoffice em React com Module Federation	Um único código para Android e iOS; microfrontends exigem composição em tempo de execução	Câmera e biometria pelo aparelho; o portal é montado a partir de pacotes independentes
RT-6	Documentação em arc42, com diagramas C4 e UML em draw.io	Exigência da disciplina	O arquivo `.drawio` é a fonte versionada de todos os diagramas e acompanha a entrega
RT-7	Hospedagem na Microsoft Azure com orçamento acadêmico	Projeto sem verba	Preferência por planos de consumo e escala a zero (Container Apps, Functions, API Management Consumption)

2.2 Restrições Organizacionais

#	Restrição	Consequência
RO-1	Equipe fixa de 4 integrantes, estudantes em período noturno	A arquitetura precisa ser divisível em frentes independentes; cada integrante assume serviços inteiros
RO-2	Prazo do semestre letivo, com entregas parciais no AVA	Escopo priorizado: o primeiro incremento implementa os módulos de Identidade e Carteira; os demais serviços são projetados
RO-3	Mesmo sistema em todas as atividades da disciplina	Decisões tomadas agora são herdadas pelas próximas entregas
RO-4	Uso obrigatório de arc42, C4 Model, UML, DER e Software Architecture Canvas, e dos estilos Serverless, Microfrontend, BFF + Microsserviços + Database per Service, API Gateway, EDA, Clean Architecture e Vertical Slice	Cada estilo tem um lugar explícito na arquitetura (seção 4.1); Clean Architecture e Vertical Slice refletem o código do incremento

2.3 Convenções

- Código, identificadores e comentários em inglês; documentação em português; commits em Conventional Commits (`feat:`, `fix:`, `test:`, `docs:`).
- Diagramas C4 seguem a notação oficial (título, legenda, tipo e tecnologia em cada elemento, setas rotuladas com ação e tecnologia). Entre processos, as setas informam protocolo, porta e rota; entre componentes do mesmo processo, são chamadas Java em memória.
- UML segue a especificação 2.5.1 da OMG; o DER usa a notação pé de galinha. Nos diagramas que refletem o código, borda tracejada indica elemento planejado. Erros da API seguem Problem Details (RFC 9457).

3. Contexto e Escopo

Esta seção delimita a Carteira Digital em relação aos seus parceiros de comunicação: quem usa o sistema e com quais sistemas vizinhos ele troca informação. O diagrama de contexto trata a Carteira Digital como caixa preta. O interior aparece a partir da seção 4.

3.1 Contexto Negocial

O diagrama de contexto do sistema (C4 Nível 1) está na Figura 2. A tabela detalha, para cada parceiro, o que entra e o que sai do sistema.

Parceiro de comunicação	Entradas (recebe do parceiro)	Saídas (envia ao parceiro)
Cliente	Dados de cadastro, documento e selfie, ordens de PIX, pagamentos e pedidos de cartão	Saldo, extrato, comprovantes, faturas e notificações
Operador de Backoffice	Decisão de aprovação de cadastro, respostas a chamados e pedidos de bloqueio	Dossiê do cliente, histórico de transações e fila de chamados
Banco Central – PIX	Confirmação de liquidação, dados do recebedor e PIX recebidos	Consulta de chave no DICT e ordens de pagamento no SPI
Banco Parceiro	Confirmação de TED e de pagamento de boleto	Ordens de TED, pagamento de boletos e tributos
Serviço de Validação e Biometria	Resultado da validação de CPF e da comparação facial	Documento de identidade e selfie do cliente
Processadora de Cartões	Autorizações de compra e eventos de fatura (webhook)	Pedidos de emissão, ajuste de limite e bloqueio de cartão
Serviço de Mensagens	Confirmação de entrega	Conteúdo das notificações por push, e-mail e SMS

Diagrama de Contexto do Sistema – Carteira Digital

C4 Nível 1 · o sistema como caixa preta, seus usuários e os sistemas vizinhos

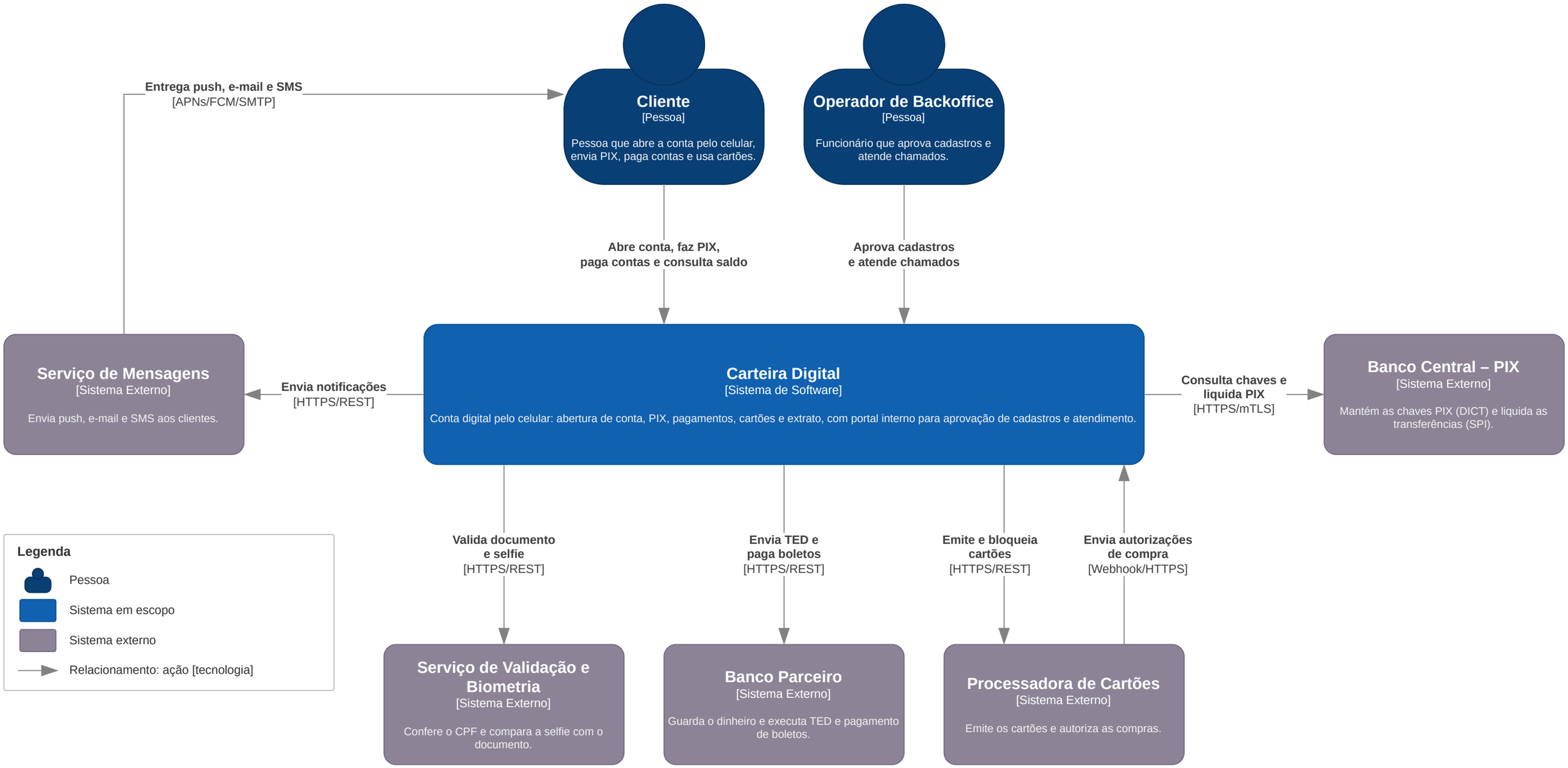


Figura 2 — Nível 1 do C4 Model: contexto do sistema Carteira Digital

3.2 Contexto Técnico

Os mesmos relacionamentos, agora descritos pelos canais técnicos. Todo tráfego externo é cifrado. A comunicação com o Banco Central exige autenticação mútua por certificado.

Canal	Protocolo, formato e porta	Autenticação	Mapeamento para o negócio
Aplicativo móvel → API Gateway (via Front Door)	HTTPS (TLS 1.3), JSON/REST, porta 443	Token JWT + PIN transacional	Todas as operações do cliente
Navegador → Static Web Apps	HTTPS, porta 443	Pública (arquivos estáticos)	Download do Shell e dos microfrontends
Portal de backoffice → API Gateway	HTTPS (TLS 1.3), JSON/REST, porta 443	Token JWT com perfil de operador	Aprovação de cadastro, consultas e chamados
Serviço de Pagamentos → DICT / SPI	HTTPS com mTLS, JSON, porta 443	Certificado digital do participante	Consulta de chave e liquidação do PIX
Serviço de Pagamentos → Banco Parceiro	HTTPS, JSON/REST, porta 443	Chave de API e IP permitido	TED, boletos e tributos
Serviço de Clientes → Bureau de Biometria	HTTPS, JSON + imagem, porta 443	Chave de API	Validação documental e prova de vida
Serviço de Cartões ↔ Processadora	HTTPS, JSON/REST, porta 443; webhook de retorno pelo API Gateway	Chave de API e assinatura HMAC no webhook	Emissão de cartão e autorizações
Funções de Notificação → Serviço de Mensagens	HTTPS, JSON/REST, porta 443	Chave de API	Push, e-mail e SMS
Microsserviços ↔ Barramento de Eventos	AMQP 1.0 sobre TLS (Azure Service Bus), porta 5671	Managed Identity	Eventos de negócio entre serviços
Microsserviço → seu banco	JDBC sobre TLS, porta 5432	Usuário de banco exclusivo do serviço	Persistência de cada serviço

4. Estratégia de Solução

Esta seção resume as decisões fundamentais que moldam a arquitetura, mostra onde cada estilo arquitetural exigido foi aplicado e como cada objetivo de qualidade da seção 1.2 é atingido.

4.1 Decisões Fundamentais e Estilos Arquiteturais

Estilo	Onde aparece	Por que foi adotado	Onde ver
Microfrontend	Portal de Backoffice: um Shell carrega o MFE Onboarding e o MFE Atendimento em tempo de execução (Module Federation)	Cada tela do backoffice é publicada sozinha. A equipe de onboarding muda a fila de aprovação sem reimplantar o atendimento	Figura 3, seção 7, ADR-04
API Gateway	Azure API Management como entrada única para o app, o portal, o webhook da processadora e a API pública	Concentra validação do JWT, limite de requisições e roteamento. Os serviços internos não ficam expostos	Figuras 3 e 10, ADR-03
Backend for Frontend	BFF Mobile (aplicativo) e BFF Backoffice (portal)	Cada canal recebe uma API sob medida para as suas telas. O app faz uma chamada por tela, e o portal agrega dados de vários serviços	Figura 3, ADR-03
Microsserviços + Database per Service	Seis serviços: Identidade, Clientes, Carteira, Pagamentos, Cartões e Antifraude, cada um com o seu banco PostgreSQL	Cada serviço tem dono, ciclo de deploy e escala próprios. Nenhum serviço lê o banco de outro: a comunicação é só por API ou evento	Figuras 4 e 11, ADR-01, ADR-09
Arquitetura Orientada a Eventos	Barramento de Eventos (Azure Service Bus) com eventos de negócio como `ClienteAprovado`, `PixRecebido` e `MovimentacaoRegistrada`	Desacopla os serviços no tempo: quem publica não espera quem consome. Falhas de um consumidor não derrubam o fluxo do cliente	Figura 5, seção 6.4, ADR-05
Serverless	Funções de Notificação (Azure Functions), disparadas por evento	Notificação tem carga em picos e fica ociosa no resto do tempo. Função cobra por execução e escala sozinha	Figuras 5 e 10, ADR-06
Clean Architecture + Vertical Slice	Dentro de cada serviço. No código do incremento, os módulos `users` e `wallets` têm uma fatia por caso de uso e o domínio no centro, sem framework	Cada caso de uso é isolado e testável; regras de negócio não dependem de banco nem de HTTP. É a base da manutenibilidade	Figuras 6 a 9, ADR-02

Decomposição em serviços

Os onze módulos funcionais da seção 1.1 foram agrupados em seis microsserviços por contexto de negócio: quem é a pessoa (Identidade), o que se sabe dela (Clientes), quanto dinheiro ela tem (Carteira), como o dinheiro sai e entra de outras instituições (Pagamentos), seus cartões (Cartões) e o risco das operações (Antifraude). Notificações viraram funções serverless e o backoffice virou um portal de microfrontends. A justificativa está na ADR-01.

Como os objetivos de qualidade são atingidos

Objetivo	Abordagem	Como se materializa
Segurança	Defesa em camadas e segredo fora do código	TLS em todo tráfego, JWT validado no API Gateway e de novo em cada serviço, checagem de dono da carteira em todo caso de uso, PIN para mover dinheiro, mTLS com o Banco Central e segredos no Azure Key Vault
Integridade financeira	Agregado com regras no domínio, transação local e bloqueio otimista	O saldo só muda pelos métodos do agregado `Wallet`; carteira e lançamentos são gravados na mesma transação; versão da linha impede gasto duplo; entre serviços, outbox e consumidores idempotentes
Manutenibilidade	Serviços independentes e fatias verticais	Um serviço por contexto, uma fatia por caso de uso, domínio sem framework e 179 testes automatizados no código do incremento
Escalabilidade elástica	Escala por serviço e computação por evento	Container Apps com réplicas de 0 a N por serviço; Functions disparadas pela fila; o barramento absorve picos

Objetivo	Abordagem	Como se materializa
Usabilidade	Respostas prontas para a tela e trabalho pesado fora do caminho do usuário	O BFF entrega a tela montada em uma chamada; notificação, antifraude de monitoramento e conciliação acontecem por evento

4.2 Diagramas de Contêineres (C4 Nível 2)

O nível 2 abre a caixa preta da seção 3 e mostra a arquitetura-alvo. Como o sistema tem cerca de trinta contêineres, o diagrama foi dividido em três vistas do mesmo sistema, cada uma focada em um conjunto de estilos. Os mesmos contêineres aparecem com o mesmo nome e a mesma descrição nas três. Cada relacionamento informa a ação, o protocolo, a porta e a rota ou o endereço chamado.

- **Figura 3 – canais, API Gateway e BFFs:** leia de cima para baixo. O cliente usa o aplicativo; o operador usa o Shell, que carrega os microfrontends. Tudo passa pelo API Gateway (`api.carteiradigital.com.br`, porta 443), que entrega a chamada ao BFF do canal pela rede privada. O BFF Mobile entra pela esquerda dos serviços e o BFF Backoffice pela direita.
- **Figura 4 – microsserviços, dados e integrações:** cada serviço tem o seu banco logo abaixo dele (JDBC sobre TLS, porta 5432). As únicas chamadas síncronas entre serviços são as do Pagamentos (análise de risco e reserva de saldo). Acima ficam os parceiros externos de cada serviço.
- **Figura 5 – eventos e serverless:** setas que entram no barramento são publicações e setas que saem são entregas a quem assina (AMQP 1.0, porta 5671). O barramento é desenhado como fila (cilindro deitado), e não como banco. As Funções de Notificação são disparadas pelo barramento e falam com o Serviço de Mensagens.

Convenção de endereços da arquitetura-alvo: cada BFF e microsserviço é um app do Azure Container Apps chamado pelo nome (por exemplo, `svc-carteira`, em que *svc* abrevia *service*), com ingress interno na porta 443 encaminhado à porta 8080 do contêiner.

4.2 Diagrama de Contêineres (C4 Nível 2) – vista 1/3

Diagrama de Contêineres – Carteira Digital (1/3): canais, API Gateway e BFFs

C4 Nível 2 · Microfrontends, API Gateway e Backend for Frontend até os microsserviços (bancos na vista 2/3, eventos na 3/3)

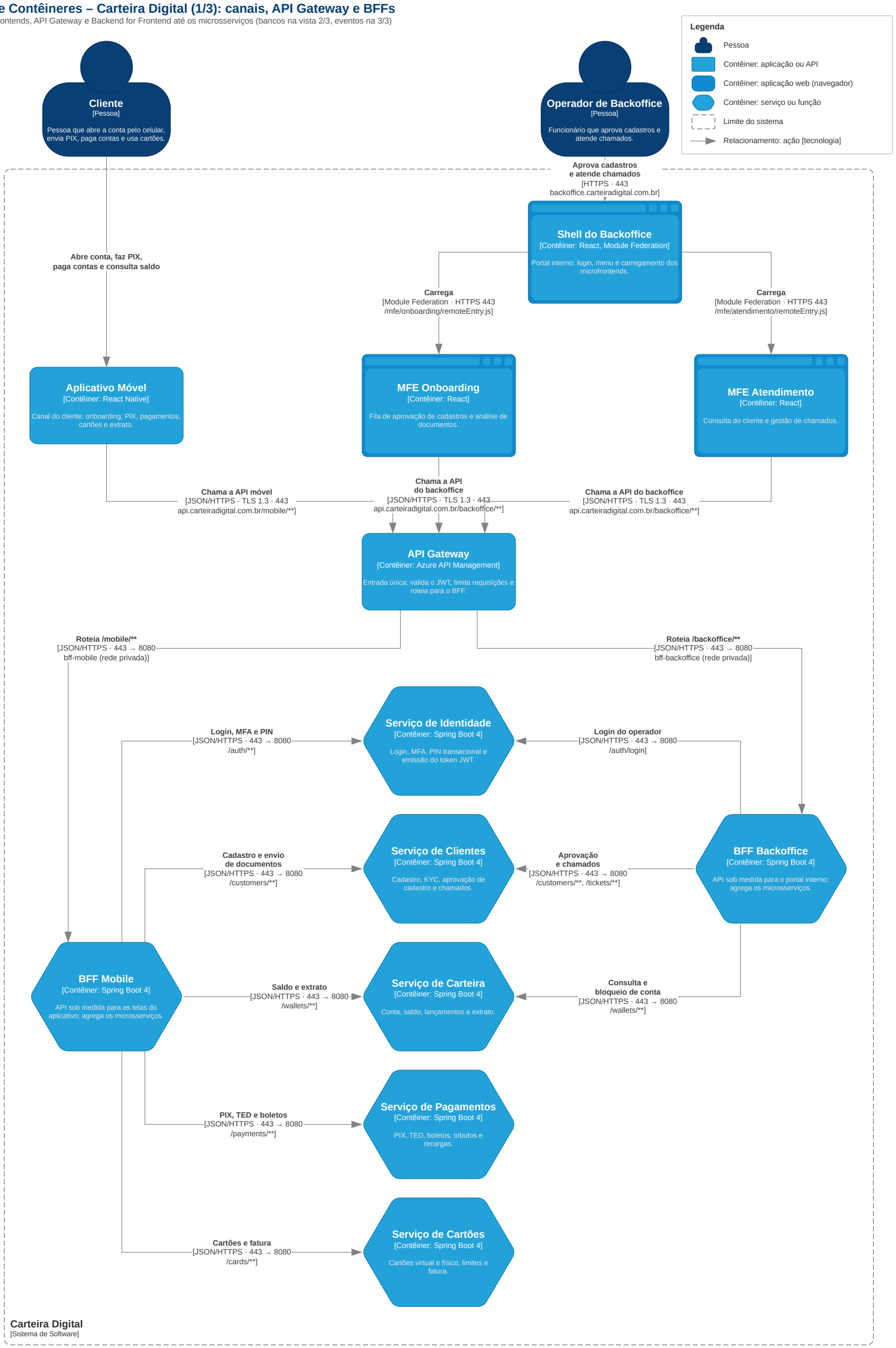


Figura 3 — Nível 2 do C4 Model (1/3): canais, API Gateway e BFFs

4.2 Diagrama de Contêineres (C4 Nível 2) – vista 2/3

Diagrama de Contêineres – Carteira Digital (2/3): microsserviços, dados e integrações

C4 Nível 2 · Database per Service, chamadas síncronas entre serviços e sistemas externos

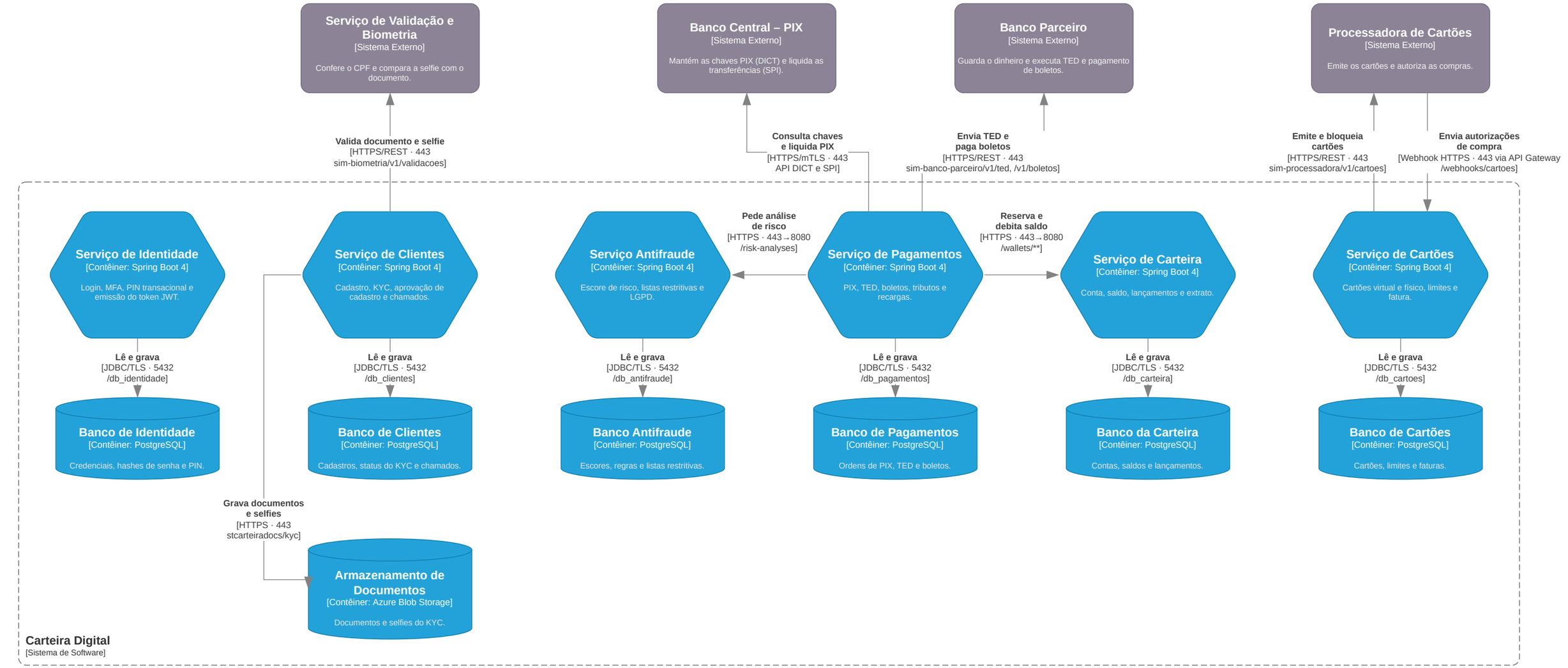


Figura 4 — Nível 2 do C4 Model (2/3): microsserviços, dados e integrações

4.2 Diagrama de Contêineres (C4 Nível 2) – vista 3/3

Diagrama de Contêineres – Carteira Digital (3/3): eventos e serverless

C4 Nível 2 · Arquitetura orientada a eventos (Azure Service Bus) e funções serverless (Azure Functions)

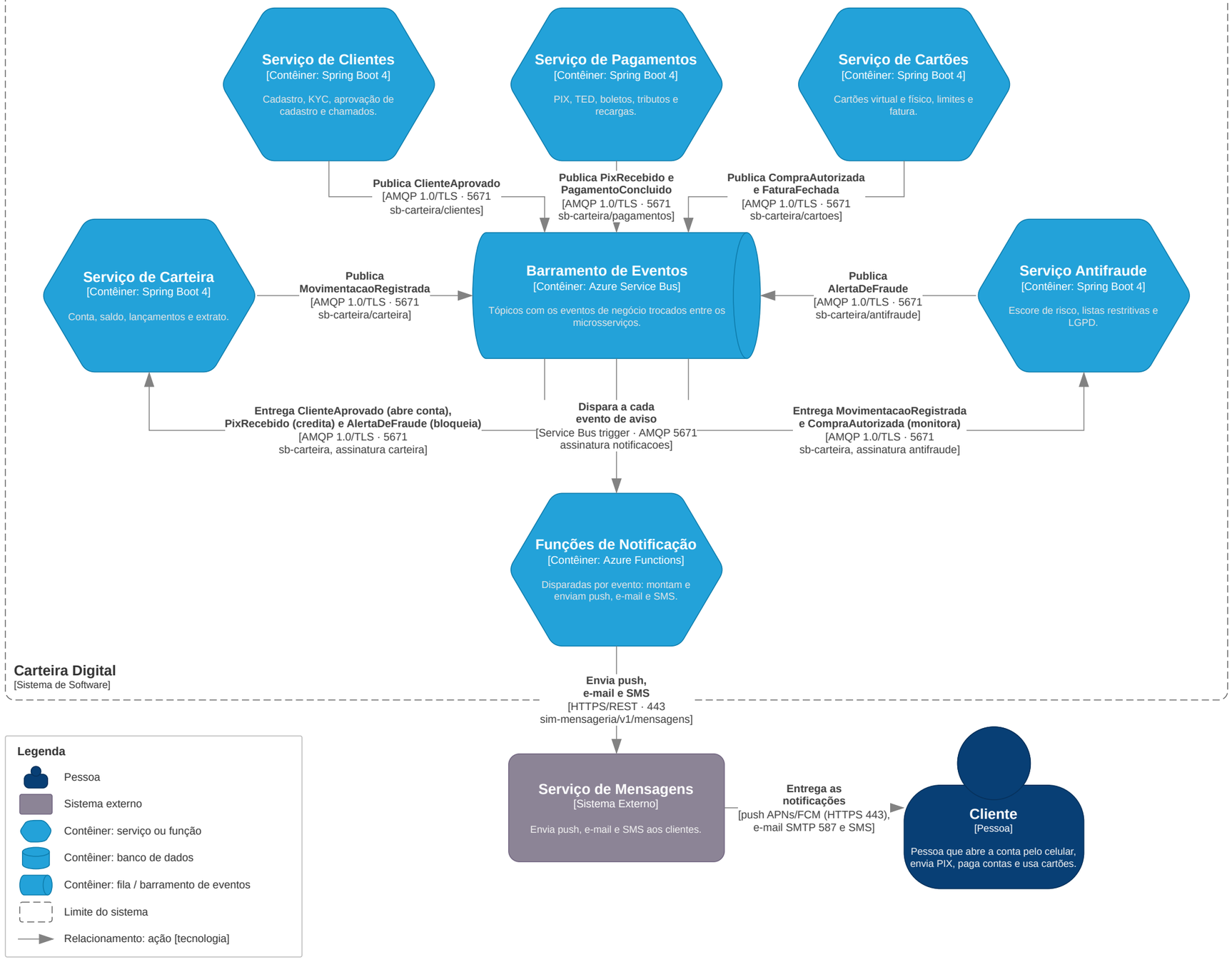


Figura 5 — Nível 2 do C4 Model (3/3): eventos e serverless

5. Visão de Blocos de Construção

Esta seção apresenta a decomposição estática do sistema. O nível 1 descreve os contêineres como caixas-pretas; o nível 2 abre o Serviço de Carteira em componentes (C4 Nível 3). O detalhamento em código da Fatia Transferir (C4 Nível 4, em UML) está na seção 6, conforme o mapeamento adotado na disciplina. O Serviço de Carteira foi escolhido porque é o mais crítico do que já está implementado: move dinheiro e precisa manter o saldo íntegro.

5.1 Visão Sistêmica Geral de Caixa Branca (contêineres)

O sistema geral está nas Figuras 3, 4 e 5 (seção 4.2). A tabela descreve a responsabilidade de cada contêiner da arquitetura-alvo e a situação dele no incremento atual.

Contêiner	Tecnologia	Responsabilidade	Requisitos	Situação
Aplicativo Móvel	React Native	Canal do cliente: onboarding, PIX, pagamentos, cartões e extrato	RF-001 a RF-057	projetado
Shell do Backoffice	React, Module Federation	Login do operador, menu e carregamento dos microfrontends	RF-065 a RF-070	projetado
MFE Onboarding	React	Fila de aprovação de cadastros e análise de documentos	RF-066	projetado
MFE Atendimento	React	Consulta do cliente e gestão de chamados	RF-065, RF-067 a RF-070	projetado
API Gateway	Azure API Management	Entrada única: valida JWT, limita requisições, roteia para os BFFs, recebe webhooks e expõe a API pública	RF-071 a RF-075	projetado
BFF Mobile	Java 25, Spring Boot 4	API sob medida para as telas do aplicativo; agrega os microsserviços	RF-001 a RF-057	projetado
BFF Backoffice	Java 25, Spring Boot 4	API sob medida para o portal interno	RF-065 a RF-070	projetado
Serviço de Identidade	Java 25, Spring Boot 4	Cadastro de credenciais, login, MFA, PIN transacional e emissão do JWT	RF-010 a RF-018	implementado módulo `users`: cadastro e login
Serviço de Clientes	Java 25, Spring Boot 4	Cadastro, KYC, aprovação de cadastro e chamados	RF-001 a RF-009, RF-065 a RF-070	projetado
Serviço de Carteira	Java 25, Spring Boot 4	Conta, saldo, lançamentos e extrato	RF-019 a RF-025, RF-050 a RF-054	implementado módulo `wallets`: 6 casos de uso
Serviço de Pagamentos	Java 25, Spring Boot 4	PIX, TED, boletos, tributos e recargas	RF-026 a RF-042	projetado
Serviço de Cartões	Java 25, Spring Boot 4	Cartões virtual e físico, limites e fatura	RF-043 a RF-049	projetado
Serviço Antifraude	Java 25, Spring Boot 4	Escore de risco, listas restritivas e LGPD	RF-058 a RF-064	projetado
Funções de Notificação	Azure Functions	Disparadas por evento: montam e enviam push, e-mail e SMS	RF-055 a RF-057	projetado
Barramento de Eventos	Azure Service Bus	Tópicos com os eventos de negócio entre serviços	RF-055 a RF-064	projetado
Bancos dos serviços (6)	PostgreSQL	Um banco por serviço; só o dono acessa	—	projetado no incremento atual, as tabelas de Identidade e Carteira rodam em H2 em memória
Armazenamento de Documentos	Azure Blob Storage	Documentos e selfies do KYC, em contêiner privado	RF-003 a RF-006	projetado

No incremento atual, os módulos `users` e `wallets` são empacotados juntos em um único executável Spring Boot. Como nenhum deles importa o outro, cada um vira um contêiner próprio sem reescrever código (risco R-1).

Interfaces entre contêineres: catálogo de eventos projetado

Evento	Publicado por	Consumido por	Efeito no consumidor
`ClienteAprovado`	Serviço de Clientes	Serviço de Carteira, Funções de Notificação	Abre a carteira do cliente; avisa que a conta foi aprovada
`PixRecebido`	Serviço de Pagamentos	Serviço de Carteira, Funções de Notificação	Credita o valor (idempotente pelo e2eId); avisa o recebedor
`PagamentoConcluido`	Serviço de Pagamentos	Funções de Notificação	Envia o comprovante
`MovimentacaoRegistrada`	Serviço de Carteira	Serviço Antifraude, Funções de Notificação	Monitora padrões suspeitos; avisa cada movimentação
`CompraAutorizada`, `FaturaFechada`	Serviço de Cartões	Serviço Antifraude, Funções de Notificação	Monitora compras; avisa a fatura
`AlertaDeFraude`	Serviço Antifraude	Serviço de Carteira	Bloqueia a conta até análise do backoffice

5.2 Nível 2 – Caixa Branca do Serviço de Carteira (C4 Nível 3)

A Figura 6 abre o Serviço de Carteira em componentes e é o diagrama que mostra os estilos Clean Architecture e Vertical Slice como estão no código do módulo `wallets`:

- **Vertical Slice.** Cada caso de uso é uma fatia com pasta própria (`create`, `deposit`, `withdraw`, `transfer`, `balance`, `statement`). A fatia contém seu controller, os objetos de entrada, a interface do caso de uso, o interactor que o implementa e a classe `*Config` que o registra. Uma fatia nunca chama outra.
- **Clean Architecture.** O domínio (`Wallet`, `Money`, `Transaction` e as portas `WalletRepository` e `TransactionHistory`) fica no centro e não importa nenhum framework. As setas apontam sempre para o domínio: as fatias o usam, e o adaptador de persistência implementa as suas portas.
- **Elementos tracejados são planejados:** a Integração com Eventos (o código ainda não publica nem consome eventos), os dois BFFs que chamarão a Carteira e o Barramento de Eventos. O Banco da Carteira é PostgreSQL na arquitetura-alvo; no incremento atual, o código usa H2 em memória com o mesmo esquema.

5.2.1 Caixas-pretas dos componentes

Componente	Responsabilidade	No código
Filtro de Segurança JWT	Valida o token (HS256, expiração) e entrega ao controller o usuário do campo `sub`; sem token válido responde 401	`shared.security.SecurityConfig`
Fatia Abrir Carteira	`POST /wallets`; um usuário tem no máximo uma carteira	`wallets.create`
Fatia Depositar	`POST /wallets/{id}/deposits`; valor positivo com até 2 casas	`wallets.deposit`
Fatia Sacar	`POST /wallets/{id}/withdrawals`; recusa saldo insuficiente	`wallets.withdraw`
Fatia Transferir	`POST /wallets/{id}/transfers`; qualquer carteira pode receber, só o dono envia	`wallets.transfer`
Fatia Consultar Saldo	`GET /wallets/{id}/balance`	`wallets.balance`
Fatia Consultar Extrato	`GET /wallets/{id}/transactions`, do mais recente ao mais antigo	`wallets.statement`
Domínio da Carteira	Regras de saldo, débito e transferência; portas de persistência; exceções de domínio	`wallets.domain`
Adaptador de Persistência	Grava carteiras e lançamentos numa transação, em ordem de id, com bloqueio otimista	`wallets.persistence`

Componente	Responsabilidade	No código
Tratador de Erros	Converte as categorias de exceção do domínio e os erros de validação em respostas RFC 9457 (400, 403, 404, 409, 422)	`shared.web.GlobalExceptionHandler`
Integração com Eventos **projetado**	Publicará `MovimentacaoRegistrada` via outbox e consumirá `ClienteAprovado`, `PixRecebido` e `AlertaDeFraude`	—

5.2.2 Organização do código do incremento

O Serviço de Identidade segue exatamente o mesmo padrão da Carteira, com as fatias `register` e `login`. Os dois módulos não importam nada um do outro: a Carteira só conhece o usuário pelo campo `sub` do JWT. Isso é o que permite separá-los em dois microsserviços sem reescrever código.

```
com.example.wallet
├── shared/
│   ├── domain/              categorias de erro (DomainException e 5 subtipos)
│   ├── security/            SecurityConfig e JwtProperties (validação do JWT)
│   └── web/                 GlobalExceptionHandler (RFC 9457)
├── users/                   → Serviço de Identidade
│   ├── domain/              User, Email, portas UserRepository e PasswordHasher
│   ├── register/            fatia POST /users
│   ├── login/               fatia POST /auth/login (porta TokenIssuer e emissor JwtTokenIssuer)
│   ├── persistence/         adaptador JPA
│   └── security/            adaptador BCrypt
└── wallets/                 → Serviço de Carteira
    ├── WalletsModuleConfig  fornece o Clock (UTC) às fatias
    ├── domain/              Wallet, Money, Transaction, exceções, portas WalletRepository e TransactionHistory
    ├── create/ deposit/ withdraw/ transfer/ balance/ statement/   uma fatia por caso de uso
    ├── persistence/         WalletPersistenceAdapter + entidades JPA
    └── web/                 TransactionResponse compartilhado pelas fatias
```

Regras que o código segue: os interactors são classes Java puras, registradas como bean por uma classe `*Config` da própria fatia; as entidades JPA são separadas do modelo de domínio e convertem com `from()` e `toDomain()`; o relógio (`Clock`) é injetado, o que torna o tempo determinístico nos testes. O emissor de token `JwtTokenIssuer` fica na fatia de login porque só ela o usa; ele implementa a porta `TokenIssuer`, e o interactor depende apenas da porta.

A Fatia Transferir é aberta no nível de código (C4 Nível 4) em três diagramas UML na seção 6: sequência (Figura 7, seção 6.1), componentes e interfaces (Figura 8, seção 6.2) e classes (Figura 9, seção 6.3).

5.2 Diagrama de Componentes (C4 Nível 3)

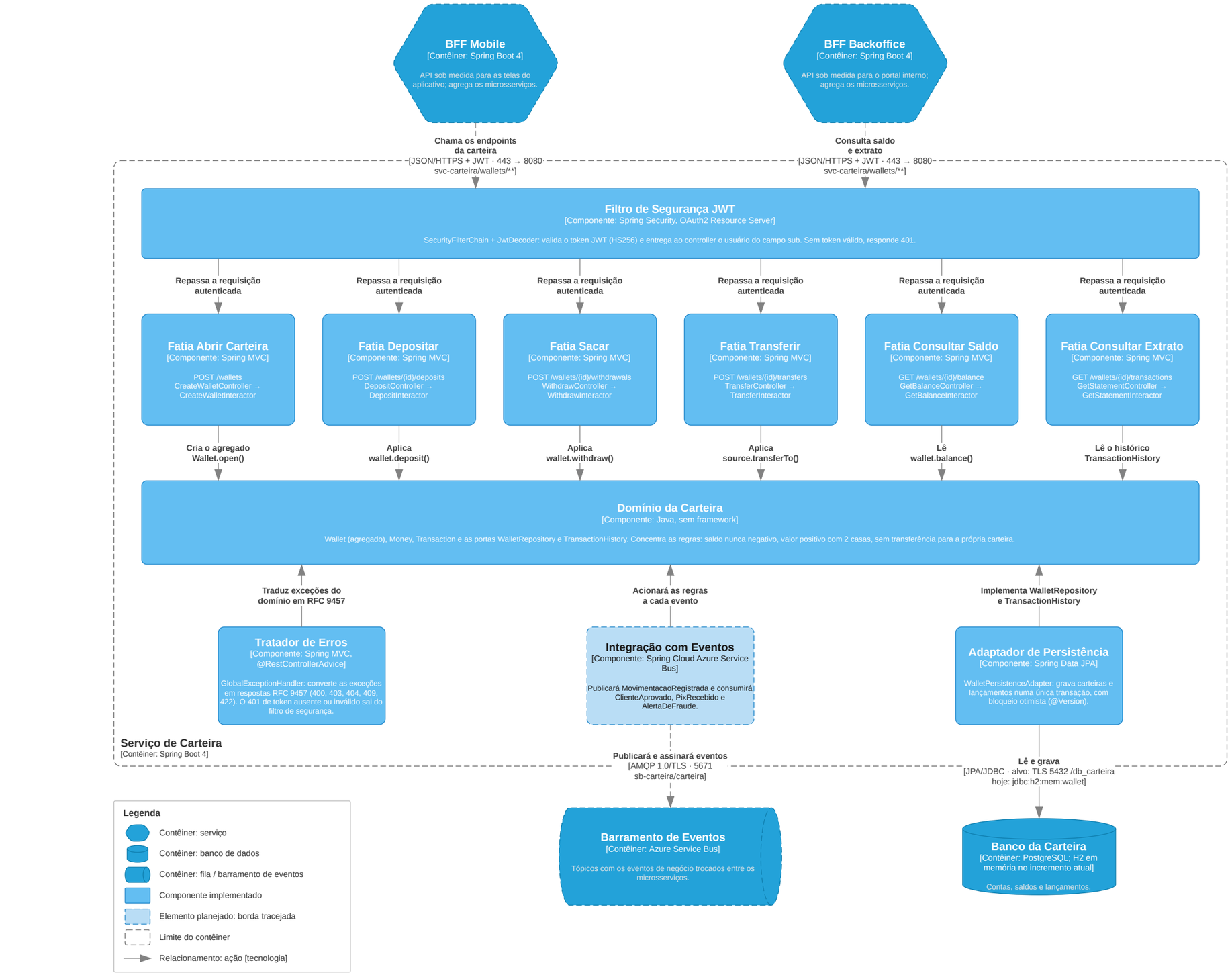


Figura 6 — Nível 3 do C4 Model: componentes do Serviço de Carteira

6. Visão de Tempo de Execução

Esta seção descreve o comportamento do sistema em execução. Seguindo o mapeamento arc42 × C4 Model adotado na disciplina, em que o diagrama de código fica na seção 6, os três diagramas de nível 4 do C4 Model, em UML, estão aqui: sequência (6.1), componentes (6.2) e classes (6.3). Os três detalham a Fatia Transferir do Serviço de Carteira, que está implementada: os diagramas mostram o que o código faz, não uma intenção. As seções 6.4 e 6.5 descrevem cenários projetados que exercitam os demais estilos.

6.1 Cenário: transferência entre carteiras implementado

É o cenário mais representativo do código do incremento: atravessa autenticação, o caso de uso, o agregado e a persistência, e é onde os objetivos de segurança e integridade financeira são postos à prova ao mesmo tempo. A ordem das etapas não é acidental: o agregado confere que destino e origem são carteiras diferentes e calcula o saldo do destino antes de debitar a origem, então qualquer falha acontece antes de alguma coisa mudar; e as duas carteiras são gravadas juntas, em ordem de id. A Figura 7 mostra o fluxo em UML.

6.1.1 Leitura do cenário

Passos	O que acontece	Por que importa
1 a 3	O filtro de segurança valida assinatura e expiração do JWT; em seguida o Spring MVC valida o corpo da requisição (`@Valid`) e chama o controller	Nenhuma movimentação começa sem identidade comprovada (objetivo 1)
4	O controller monta o comando com o usuário do campo `sub` do token	O requisitante vem do token, nunca do corpo da requisição
5 a 10	O interactor carrega a origem, confere que o requisitante é o dono e carrega o destino. Cada leitura roda numa transação somente leitura, separada da gravação	Só o dono envia; qualquer carteira pode receber (403 e 404 nos casos de erro)
11 a 15	O interactor converte o valor em `Money` e chama `transferTo`, que confere destino ≠ origem, calcula o novo saldo do destino, debita a origem e registra TRANSFER_IN e TRANSFER_OUT	Valor com mais de 2 casas (`Money.of`, no passo 11), saldo insuficiente ou transferência para a própria carteira: 422. Valor nulo, zero ou negativo já foi recusado com 400 no passo 3 (`@Positive`)
16 e 17	O adaptador grava as duas carteiras e os dois lançamentos numa única transação, em ordem de id, checando a versão	Saldo e extrato mudam juntos; gravação concorrente vira 409 e não gasta o mesmo dinheiro duas vezes (objetivo 2)
18 a 20	A transação TRANSFER_OUT volta como 201 Created	O cliente recebe o comprovante da operação
break	Se uma regra falha (passos 11 e 12) ou a versão está desatualizada (passo 17), a exceção sobe linha de vida a linha de vida até o controller, e o Spring MVC a entrega ao tratador, que responde 422 ou 409	O erro chega ao cliente no formato Problem Details e nada fica gravado pela metade

No diagrama, o BFF Mobile aparece tracejado porque ainda não existe: no incremento atual, quem chama o endpoint é qualquer cliente HTTP com um token válido. O Banco da Carteira é PostgreSQL na arquitetura-alvo e H2 em memória no incremento.

6.1 Diagrama de Sequência UML (C4 Nível 4)

Diagrama de Sequência UML – transferência entre carteiras (POST /wallets/{id}/transfers)

C4 Nível 4 · comportamento do código real da Fatia Transferir, do token JWT até a gravação atômica das duas carteiras

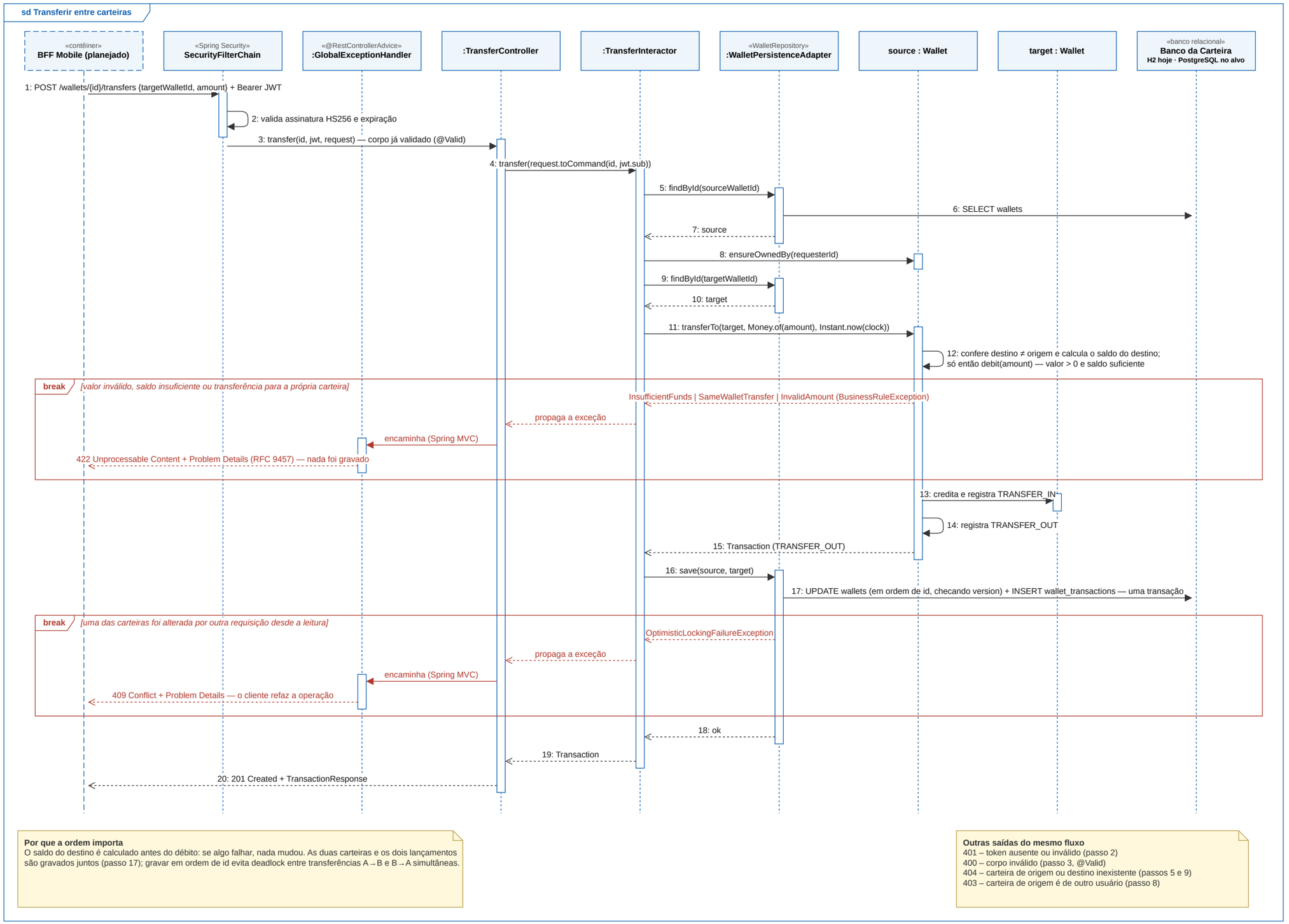


Figura 7 — Nível 4 do C4 Model em UML: sequência da transferência entre carteiras

6.2 Componentes e interfaces da Fatia Transferir (UML)

A Figura 8 detalha, em notação UML de componentes, as interfaces que a Fatia Transferir oferece e exige. A bola representa uma interface oferecida; a seta tracejada «use» representa quem a exige. A leitura importante é a da porta `WalletRepository`: ela pertence ao domínio e é implementada pelo adaptador de persistência. O interactor não conhece JPA nem banco, e o domínio não depende de nenhuma outra camada. Essa é a inversão de dependência da Clean Architecture.

6.2.1 Interfaces

Interface	Oferecida por	Operações	Consumida por
`POST /wallets/{id}/transfers` (HTTP na porta 8080 do contêiner; o TLS termina no ingress 443)	`TransferController`	Corpo `{targetWalletId, amount}`; responde 201 com a transação TRANSFER_OUT	BFF Mobile **projetado**; hoje, qualquer cliente HTTP autenticado
`TransferUseCase`	`TransferInteractor`	`transfer(TransferCommand): Transaction`	`TransferController`
`WalletRepository`	`WalletPersistenceAdapter`	`findById`, `existsByOwnerId`, `save(Wallet...)`	Interactors de todas as fatias
`TransactionHistory`	`WalletPersistenceAdapter`	`findByWalletId` (mais recente primeiro)	Fatia Consultar Extrato
`Clock`	`WalletsModuleConfig`	Instante atual em UTC, com precisão de microssegundos	Interactors que gravam lançamentos
`Jwt`	Spring Security	Usuário autenticado (`sub`)	Controllers

A ligação entre interface e implementação é feita pela classe `TransferConfig` da própria fatia, que cria o `TransferInteractor` com o `WalletRepository` e o `Clock` (ver Figura 9).

6.2 Diagrama de Componentes UML (C4 Nível 4)

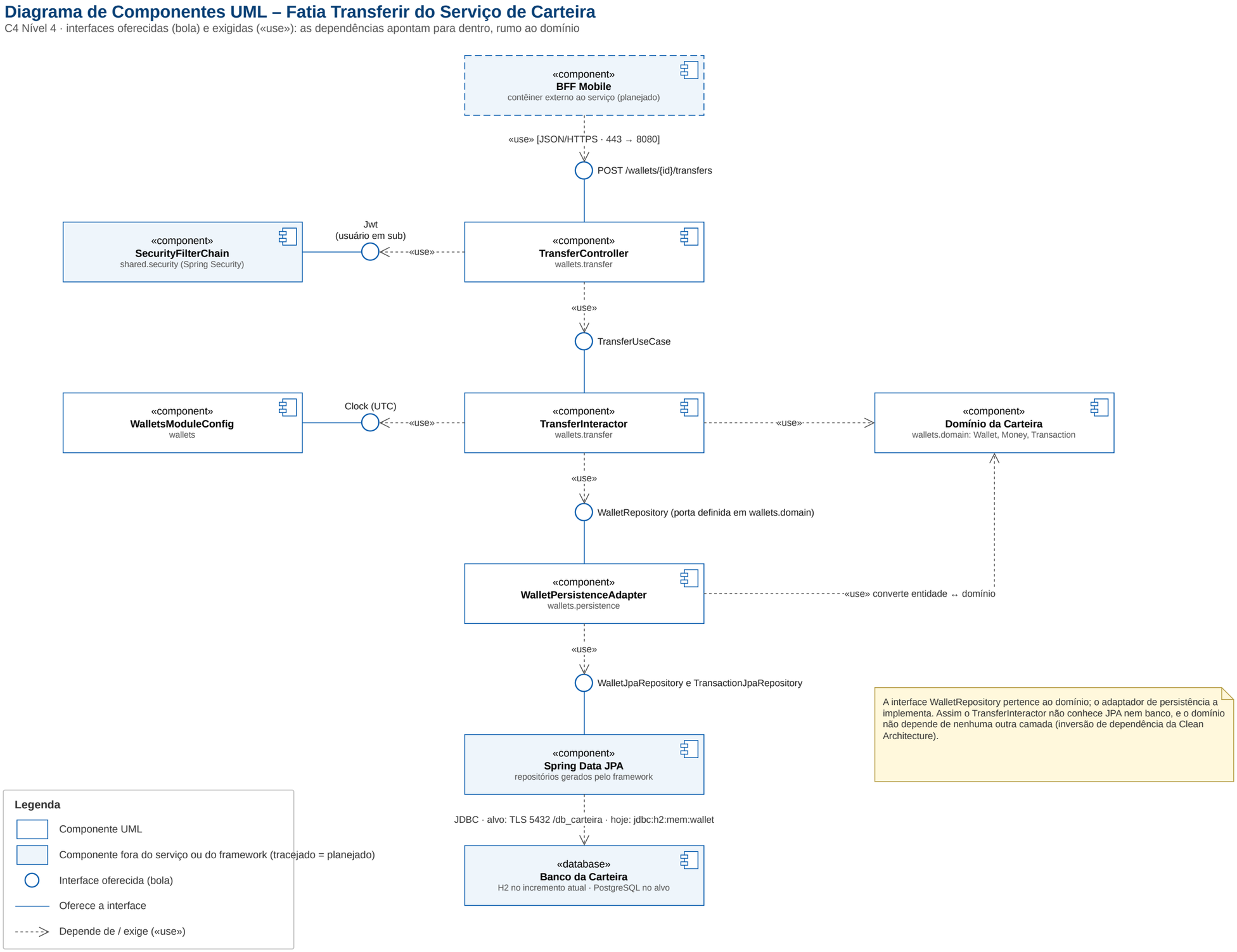


Figura 8 — Nível 4 do C4 Model em UML: componentes e interfaces da Fatia Transferir

6.3 Modelo de domínio em execução — Diagrama de Classes (UML)

A Figura 9 apresenta as classes reais que participam do cenário da seção 6.1, agrupadas por pacote. Quatro pontos merecem destaque:

- **Wallet é o agregado.** O saldo nunca é atribuído de fora: ele só muda por `deposit`, `withdraw` e `transferTo`. Cada operação acumula um `Transaction` na lista de pendentes (composição), que o repositório grava junto com a carteira.
- **Money é um valor.** Nunca negativo, sempre com escala 2 e no máximo 17 dígitos inteiros, o que corresponde à coluna `numeric(19,2)`. Valores inválidos são recusados na construção.
- **As regras viram exceções de domínio.** `WalletNotFoundException`, `WalletAccessDeniedException`, `InsufficientFundsException`, `SameWalletTransferException` e `InvalidAmountException` estendem as categorias de `shared.domain`, e o tratador de erros traduz cada categoria no status HTTP (seção 8.4).
- **Nenhuma seta sai de `wallets.domain`.** A fatia e o adaptador dependem do domínio; as entidades JPA ficam no pacote de persistência, têm visibilidade de pacote (~) e só convertem de e para o domínio.

6.3 Diagrama de Classes UML (C4 Nível 4)

Diagrama de Classes UML – Fatia Transferir e domínio da Carteira

C4 Nível 4 · classes reais do código, agrupadas por pacote: a fatia e o adaptador dependem do domínio; o domínio não depende de ninguém

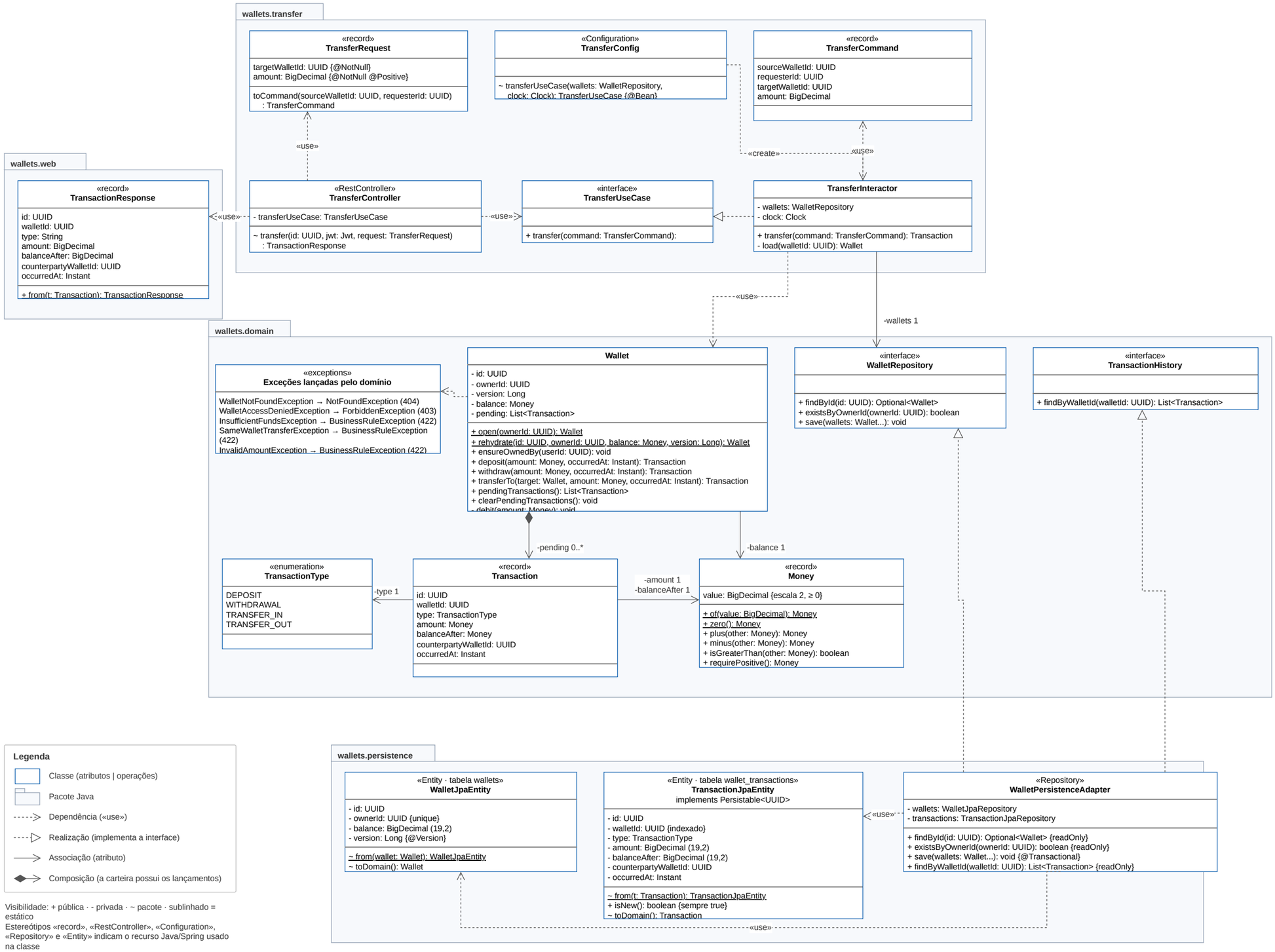


Figura 9 — Nível 4 do C4 Model em UML: classes da Fatia Transferir e do domínio da Carteira

6.4 Cenário: PIX recebido, de ponta a ponta por eventos projetado

Este cenário mostra a arquitetura orientada a eventos e o serverless trabalhando juntos (Figura 5).

#	O que acontece	Estilo envolvido
1	O SPI (simulador) informa ao Serviço de Pagamentos que chegou um PIX para uma chave do cliente	Integração externa
2	Pagamentos grava o pagamento e o evento `PixRecebido` na tabela de outbox, na mesma transação do seu banco	Database per Service, outbox
3	O publicador do outbox envia o evento ao Barramento de Eventos (AMQP 1.0, porta 5671)	EDA
4	A Carteira consome o evento e credita o valor; o e2eId do PIX garante que um evento repetido não credite duas vezes	EDA, consumidor idempotente
5	A Carteira publica `MovimentacaoRegistrada`	EDA
6	O Antifraude consome o evento e atualiza o perfil de risco; as Funções de Notificação são disparadas e enviam o push ao recebedor	EDA, Serverless

6.5 Cenário: aprovação de cadastro no backoffice projetado

#	O que acontece	Estilo envolvido
1	O operador abre o portal; o Shell carrega o MFE Onboarding em tempo de execução	Microfrontend
2	O MFE chama o API Gateway, que valida o token de operador e roteia para o BFF Backoffice	API Gateway, BFF
3	O BFF Backoffice pede ao Serviço de Clientes a aprovação do cadastro	Microsserviços
4	Clientes grava a aprovação e publica `ClienteAprovado`	EDA
5	A Carteira consome o evento e abre a carteira; as Funções de Notificação avisam o cliente	EDA, Serverless

7. Visão de Implantação

Esta seção descreve a infraestrutura em que a arquitetura-alvo será executada e o mapeamento dos contêineres sobre ela. Toda a implantação descrita aqui é **projetado**: o incremento atual roda localmente, como um único executável com banco em memória. A nuvem é a Microsoft Azure, região Brazil South, para manter a latência baixa em relação aos usuários e aos parceiros nacionais. Cada recurso foi escolhido por ter plano de consumo ou camada de entrada compatível com a restrição RT-7.

7.1 Nível de Infraestrutura 1

Motivação. Cada estilo arquitetural tem um serviço gerenciado que o implementa diretamente: Static Web Apps publica cada microfrontend de forma independente; API Management é o API Gateway; Container Apps executa BFFs e microsserviços como contêineres com escala própria; Service Bus é o barramento de eventos; Functions é a computação serverless; e o PostgreSQL Flexible Server hospeda um banco por serviço.

Características de qualidade e desempenho. Container Apps escala cada serviço de 0 a N réplicas por número de requisições, e as Functions escalam pelo tamanho da fila. Isso atende ao objetivo de escalabilidade elástica e ao orçamento. O Service Bus guarda os eventos enquanto um consumidor estiver fora do ar, com reentrega e fila de mensagens mortas. O preço da escala a zero é a latência de partida a frio na primeira chamada após ociosidade, registrada como risco na seção 11.

Mapeamento de blocos de construção para infraestrutura.

Contêiner	Serviço Azure	Plano / escala	Justificativa
Aplicativo Móvel	Lojas Apple e Google (fora da Azure)	—	Roda no aparelho do cliente
Shell e microfrontends	Azure Static Web Apps	Camada gratuita, CDN	Cada MFE é um pacote estático publicado sozinho
Ponto de entrada	Azure Front Door	CDN + WAF	Termina o TLS e filtra ataques antes do gateway
API Gateway	Azure API Management	Consumption	Paga por chamada; políticas de JWT e de limite de requisições
BFFs e microsserviços	Azure Container Apps	Consumo, 0 a N réplicas por serviço	Contêiner Docker sem administrar servidor; cada serviço escala sozinho
Funções de Notificação	Azure Functions	Consumption, gatilho do Service Bus	Cobrança por execução
Barramento de Eventos	Azure Service Bus	Standard (tópicos e assinaturas)	Publicação para vários assinantes, reentrega e fila de mensagens mortas
Bancos dos serviços	Azure Database for PostgreSQL – Flexible Server	Burstable, um servidor com seis bancos	ACID gerenciado, backup diário; isolamento por banco e por usuário (ADR-09)
Armazenamento de Documentos	Azure Storage Account (Blob)	Redundância local, contêiner privado	Arquivos grandes fora do banco
Segredos	Azure Key Vault	Standard	Segredo do JWT, certificados mTLS e chaves de parceiros fora do código
Observabilidade	Azure Monitor / Application Insights	Por volume ingerido	Logs, métricas e traces de todos os contêineres
Simuladores de parceiros	Azure Container Apps (ambiente separado)	Consumo	Atende RT-1 e RT-2 sem tocar no código dos serviços

7.1 Diagrama de Implantação

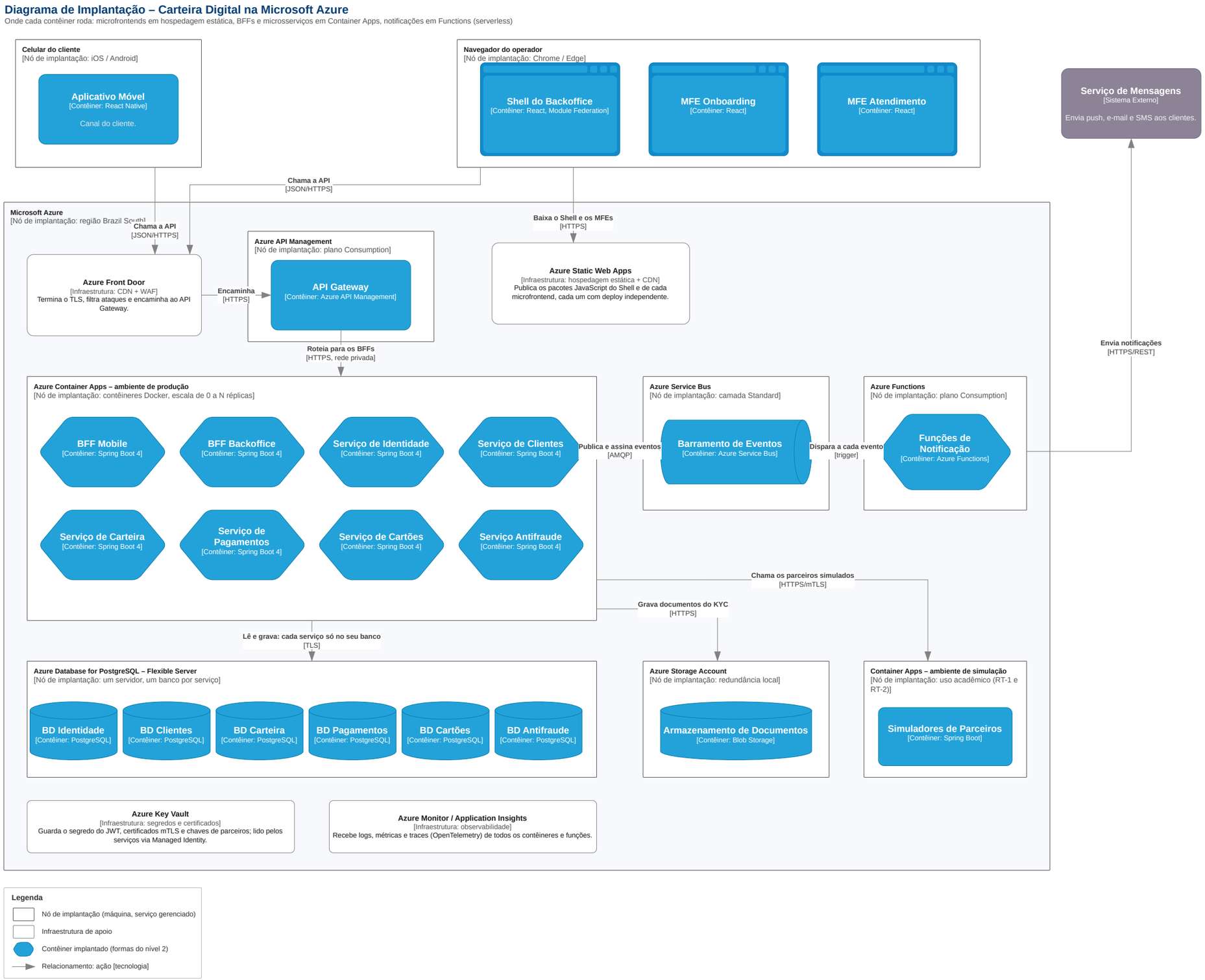


Figura 10 — Implantação da Carteira Digital na Microsoft Azure (arquitetura-alvo)

7.2 Nível de Infraestrutura 2

Azure Container Apps – ambiente de produção

Os oito contêineres da arquitetura-alvo (dois BFFs e seis microsserviços) ficam no mesmo ambiente, numa rede privada. Só o API Gateway alcança os BFFs; os microsserviços aceitam apenas tráfego interno, no ingress interno da porta 443, encaminhado à porta 8080 de cada contêiner. Cada serviço tem o próprio pipeline de build e a própria imagem, e acessa banco, barramento e cofre com Managed Identity, sem senha em arquivo de configuração.

Azure Database for PostgreSQL – Flexible Server

Um único servidor hospeda seis bancos, um por serviço, cada um com um usuário que só tem permissão no próprio banco (porta 5432, TLS obrigatório). O isolamento de Database per Service é garantido no nível lógico, que é o que importa para o acoplamento; separar servidores é uma evolução prevista se a carga de um serviço exigir.

Azure Static Web Apps e Module Federation

O Shell e cada microfrontend são publicados como pacotes separados. O Shell conhece só o endereço do `remoteEntry.js` de cada MFE e os carrega em tempo de execução; publicar uma nova versão do MFE Onboarding não exige republicar o Shell nem o MFE Atendimento.

8. Conceitos Transversais

Regras e soluções que valem para várias partes do sistema e que, por isso, não pertencem a um único bloco de construção.

8.1 Modelo de Dados

O modelo da Figura 11 aplica Database per Service: cada moldura é o banco de um serviço, e só esse serviço o lê e escreve. Dentro de um banco, as relações usam chave estrangeira normalmente. Entre bancos não existe chave estrangeira: uma coluna marcada **REF** guarda o identificador de um dado que pertence a outro serviço — por exemplo, `wallets.owner_id` guarda o `users.id` do Serviço de Identidade. As tabelas `users`, `wallets` e `wallet_transactions` são as que o código do incremento cria (hoje em H2 em memória; o mesmo esquema vale para o PostgreSQL da arquitetura-alvo); as demais, tracejadas no diagrama, são projetadas.

A notação é a pé de galinha (Information Engineering). O símbolo em cada ponta diz a cardinalidade daquele lado: traço é exatamente um, círculo é opcional e o pé de galinha é muitos. Linha cheia indica relacionamento identificador, em que a chave estrangeira faz parte da chave primária do filho (por exemplo, `transacao_pix.pagamento_id`); linha tracejada indica relacionamento não identificador.

8.1.1 Decisões de modelagem

Decisão	Motivo
Valores em `numeric(19,2)`	Aritmética decimal exata; ponto flutuante introduziria erro de arredondamento em dinheiro. O tipo `Money` do domínio garante o mesmo limite
Saldo armazenado em `wallets` e cada mudança registrada em `wallet_transactions` com `balance_after`	Leitura rápida do saldo, e o extrato é consequência natural do modelo: cada lançamento diz como o saldo ficou
Coluna `version` em `wallets`	Bloqueio otimista: uma gravação com versão desatualizada falha e vira 409
`owner_id` único	Um usuário tem no máximo uma carteira, garantido também pelo banco
Chaves primárias em UUID, geradas pelo domínio	Identificador não sequencial, que não revela volume nem permite varredura; os lançamentos são sempre inseridos, nunca atualizados
Sem chave estrangeira entre bancos (REF)	Database per Service: cada serviço evolui o seu esquema sem coordenar com os outros
`outbox_eventos` em cada banco que publica eventos (o DER a desenha no BD Carteira; Clientes, Pagamentos, Cartões e Antifraude têm a mesma tabela)	O evento é gravado na mesma transação da mudança de estado; nada é publicado sem ter sido gravado, nem gravado sem ser publicado
`transacao_pix.e2e_id` e `pagamento.chave_idempotencia` únicos	Impedem registrar a mesma liquidação ou o mesmo pedido duas vezes

8.1 Diagrama de Entidades e Relacionamentos

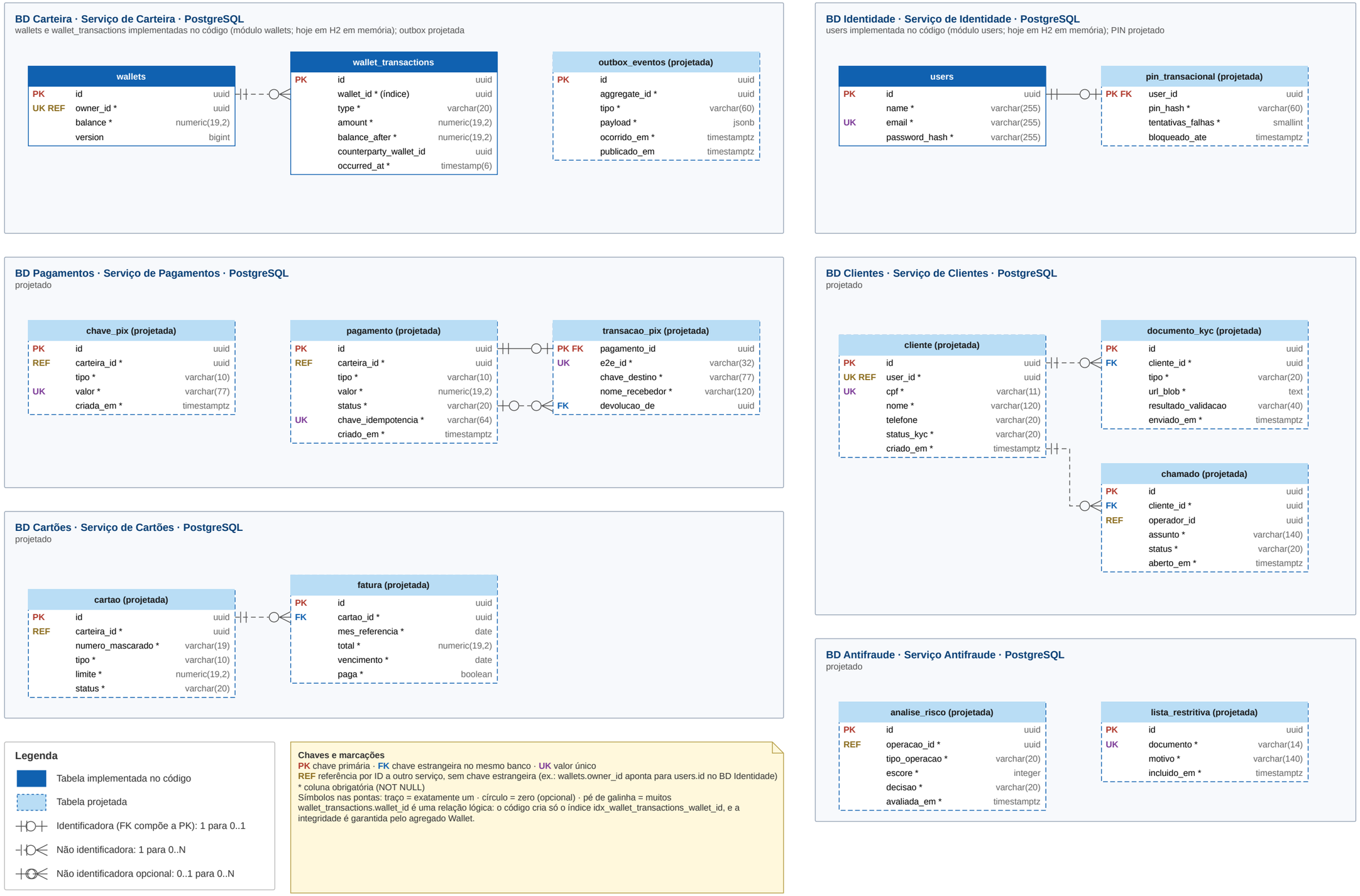


Figura 11 — Modelo de dados da Carteira Digital em notação entidade-relacionamento, um banco por serviço

8.2 Segurança

- **Autenticação.** O Serviço de Identidade emite um JWT com o id do usuário em `sub`, o e-mail e validade de 1 hora. O API Gateway valida o token na entrada e cada serviço valida de novo (Spring Security como resource server). O segredo de assinatura vem da variável `WALLET_JWT_SECRET` (há um valor padrão apenas para desenvolvimento, que precisa ser sobrescrito em qualquer ambiente real) e precisa ter pelo menos 32 bytes; com menos, a aplicação não sobe.
- **Autorização.** Todo caso de uso da Carteira confere se o requisitante é o dono da carteira antes de agir (403 caso contrário). Movimentar dinheiro também exigirá o PIN transacional projetado.
- **Senhas.** Guardadas só como hash BCrypt. O cadastro exige de 8 caracteres a 72 bytes, o limite do BCrypt, verificado por uma validação própria.
- **Sem vazamento no login.** E-mail inexistente, malformado ou senha errada recebem a mesma resposta (401), e o e-mail inexistente também é comparado contra um hash fictício, para que as duas falhas levem o mesmo tempo.
- **Logs.** Os objetos de entrada mascaram a senha no `toString()`; logs nunca registram PIN, token, número de cartão ou imagem de documento.
- **Infraestrutura.** mTLS com o Banco Central, segredos no Key Vault e Managed Identity entre serviços e recursos da Azure.

8.3 Transações e consistência

Dentro de um serviço vale a transação ACID do seu banco. Na Carteira, as leituras do caso de uso rodam em transações somente leitura, e o adaptador grava todas as carteiras alteradas e todos os lançamentos pendentes numa única transação de escrita; se qualquer parte falhar, nada é gravado. A coluna de versão garante que duas requisições concorrentes não apliquem débitos sobre o mesmo saldo, e as carteiras são gravadas em ordem de id para que transferências opostas simultâneas (A→B e B→A) não entrem em deadlock.

Entre serviços a consistência é eventual e coordenada por eventos. Três regras evitam perda e duplicidade: o evento é gravado no outbox na mesma transação da mudança; todo consumidor é idempotente (usa o id do evento ou uma chave de negócio, como o e2eId); e fluxos que envolvem mais de um serviço usam saga com compensação — no PIX enviado, por exemplo, a Carteira reserva o saldo, Pagamentos envia ao SPI e, se a liquidação falhar, a reserva é liberada em vez de o dinheiro sumir.

8.4 Tratamento de erros

As exceções de negócio formam uma hierarquia selada com cinco categorias (`shared.domain`), e o tratador `GlobalExceptionHandler` as converte em respostas no formato Problem Details (RFC 9457), junto com os erros de validação de entrada. Um módulo nunca escolhe o status HTTP: ele escolhe a categoria. A exceção é o 401 de token ausente ou inválido, respondido pelo próprio filtro do Spring Security antes de a requisição chegar a um controller, sem corpo.

Categoria / situação	Status	Quem responde	Exemplos no código
Corpo inválido (validação de entrada)	400	Tratador de erros	Valor nulo ou não positivo; resposta inclui o mapa `errors` por campo
Token ausente ou inválido	401	Filtro do Spring Security	Requisição sem `Authorization`, token expirado ou assinatura inválida
Credenciais inválidas (`UnauthorizedException`)	401	Tratador de erros	E-mail ou senha incorretos no login
Proibido (`ForbiddenException`)	403	Tratador de erros	Movimentar a carteira de outro usuário
Não encontrado (`NotFoundException`)	404	Tratador de erros	Carteira inexistente
Conflito (`ConflictException` e bloqueio otimista)	409	Tratador de erros	E-mail já cadastrado; usuário que já tem carteira; carteira alterada por outra requisição
Regra de negócio (`BusinessRuleException`)	422	Tratador de erros	Saldo insuficiente; valor com mais de 2 casas; transferência para a própria carteira

Chamadas a parceiros externos passam por tempo limite, nova tentativa com espera crescente e disjuntor projetado. Falha temporária mantém a operação pendente para nova tentativa; falha permanente encerra a operação como recusada e executa a compensação.

8.5 Comunicação por eventos projetado

- Eventos têm nome no passado, em linguagem de negócio (`PixRecebido`, não `PixUpdated`), e um envelope comum: id, tipo, versão, instante e id de correlação.
- Cada serviço publica num tópico próprio; cada consumidor tem a sua assinatura. Assim um novo consumidor não exige mudar o publicador.
- A entrega é pelo menos uma vez: por isso todo consumidor é idempotente. Mensagens que falham repetidamente vão para a fila de mensagens mortas e geram alerta.
- Mudanças de contrato são aditivas; uma mudança incompatível cria uma nova versão do evento.

8.6 Observabilidade

Cada requisição recebe um id de correlação que acompanha as chamadas síncronas e os metadados dos eventos. Logs, métricas e traces vão para o Application Insights via OpenTelemetry, o que permite reconstruir uma operação que atravessa gateway, BFF, serviços, barramento e funções.

8.7 Proteção de dados pessoais (LGPD)

A coleta é mínima: apenas o necessário para abrir e operar a conta. Documentos e selfies ficam em contêiner privado do Blob Storage, acessados por link temporário e com registro de acesso. Dados sensíveis são cifrados em repouso pelos serviços da Azure. Consultas do backoffice ao dossiê do cliente geram registro de auditoria com o identificador do operador.

8.8 Estratégia de testes

O código do incremento tem **179 testes automatizados em 37 classes** (156 métodos `@Test` e 5 testes parametrizados, que executam vários casos cada), organizados pela mesma estrutura de fatias:

Tipo	O que cobre	Como
Domínio	`Money`, `Wallet`, `Email`, `User` e as exceções	JUnit puro, sem Spring
Casos de uso	Cada interactor	Repositório em memória no lugar do banco
Web	Cada controller, validação e contrato de erro	`@WebMvcTest` (10 classes)
Persistência	Adaptadores JPA, incluindo a ordem de gravação das carteiras	`@DataJpaTest` com H2
Ponta a ponta	Jornada completa e contrato de erro com tokens reais sobre HTTP, incluindo transferências recusadas sem efeito colateral	`@SpringBootTest` com H2 (14 testes)

Na arquitetura-alvo, cada microsserviço mantém essa pirâmide e acrescenta testes de contrato para as suas APIs e para os eventos que publica, executados no pipeline de cada serviço.

9. Decisões Arquiteturais

As decisões seguem o formato de Architecture Decision Record apresentado em sala: título, contexto, decisão, estado e consequências, incluindo as negativas. Elas correspondem às hipóteses arquiteturais do Software Architecture Canvas (Figura 1).

ADR-01: Microsserviços com um banco por serviço (substitui o monolito modular da entrega 1)

Seção	Conteúdo
Contexto	A entrega 1 adotou um monolito modular para reduzir custo operacional. A disciplina passou a exigir BFF + Microsserviços + Database per Service, e os objetivos de manutenibilidade e escalabilidade se beneficiam de unidades implantadas e escaladas separadamente.
Decisão	Dividir o sistema em seis microsserviços por contexto de negócio (Identidade, Clientes, Carteira, Pagamentos, Cartões e Antifraude), cada um com o seu banco PostgreSQL, comunicando-se apenas por API ou evento.
Estado	Aceita. Substitui a ADR-01 da entrega 1. Implantação em andamento: o primeiro incremento já separa Identidade e Carteira em módulos independentes, ainda empacotados juntos (R-1).
Consequências	Positiva: deploy, escala e falhas isolados por serviço; cada integrante é dono de serviços inteiros. Negativa: consistência entre serviços passa a ser eventual, e há mais peças para operar e observar. Neutra: o código já foi escrito com módulos que não se importam entre si, o que torna a separação mecânica.

ADR-02: Clean Architecture com Vertical Slice dentro de cada serviço

Seção	Conteúdo
Contexto	Quatro pessoas trabalham em paralelo e as regras financeiras precisam ser testadas sem banco e sem HTTP.
Decisão	Organizar cada serviço por caso de uso (uma fatia por pasta, com controller, entrada, interface, interactor e configuração) em volta de um domínio sem framework. Portas no domínio; adaptadores de persistência e segurança na borda.
Estado	Aceita e implementada nos módulos `users` e `wallets`.
Consequências	Positiva: um caso de uso novo é uma pasta nova; regras testadas com JUnit puro; troca de banco não toca no domínio. Negativa: mais classes por caso de uso (comando, interface, interactor, configuração) e alguma repetição entre fatias.

ADR-03: API Gateway único e um BFF por canal

Seção	Conteúdo
Contexto	O app e o portal têm necessidades diferentes, e expor seis serviços diretamente aos clientes espalharia autenticação, limite de requisições e versionamento.
Decisão	Usar o Azure API Management como entrada única e criar o BFF Mobile e o BFF Backoffice, que agregam os microsserviços para as suas telas.
Estado	Aceita.
Consequências	Positiva: políticas de borda em um só lugar; cada canal evolui a sua API sem afetar o outro. Negativa: um salto de rede a mais por requisição e dois contêineres a mais para manter.

ADR-04: Microfrontends com Module Federation no portal de backoffice

Seção	Conteúdo
Contexto	O backoffice reúne áreas com donos diferentes (onboarding e atendimento) que mudam em ritmos diferentes.
Decisão	Um Shell em React carrega, em tempo de execução, o MFE Onboarding e o MFE Atendimento, publicados separadamente no Static Web Apps. O aplicativo móvel continua sendo um único app React Native.
Estado	Aceita.
Consequências	Positiva: deploy independente por área do portal. Negativa: é preciso manter um padrão visual comum e versões compatíveis das bibliotecas compartilhadas entre Shell e MFEs.

ADR-05: Eventos de negócio no Azure Service Bus com outbox transacional

Seção	Conteúdo
Contexto	Com um banco por serviço não há transação que abranja dois serviços. Notificar, monitorar fraude e abrir a carteira após a aprovação não precisam bloquear o usuário.
Decisão	Publicar eventos de negócio em tópicos do Service Bus, gravando-os antes no outbox da mesma transação, e exigir consumidores idempotentes.
Estado	Aceita; implementação prevista para a próxima entrega.
Consequências	Positiva: serviços desacoplados no tempo e falhas de consumidores isoladas. Negativa: consistência eventual, necessidade de idempotência e de monitorar a fila de mensagens mortas.

ADR-06: Notificações em funções serverless

Seção	Conteúdo
Contexto	Notificações chegam em picos (horário comercial, viradas de mês) e não têm estado próprio.
Decisão	Implementar as notificações como Azure Functions disparadas pelo Service Bus, substituindo o Worker Assíncrono da entrega 1.
Estado	Aceita.
Consequências	Positiva: escala automática e custo zero quando ocioso. Negativa: partida a frio, limite de tempo de execução e um modelo de deploy diferente do resto dos serviços.

ADR-07: Bloqueio otimista e gravação ordenada para o saldo

Seção	Conteúdo
Contexto	Duas requisições simultâneas podem tentar gastar o mesmo saldo, e transferências opostas simultâneas podem travar uma à outra.
Decisão	Coluna `version` na carteira (bloqueio otimista) e gravação das carteiras em ordem de id dentro de uma única transação.
Estado	Aceita e implementada.
Consequências	Positiva: impossível gastar duas vezes o mesmo saldo, sem bloquear linhas durante a leitura. Negativa: em conflito, o cliente recebe 409 e precisa repetir a operação; não há nova tentativa automática.

ADR-08: Simuladores com a mesma interface dos serviços reais

Seção	Conteúdo
Contexto	As restrições RT-1 e RT-2 impedem acesso ao SPI/DICT, ao banco liquidante, à biometria e à processadora.
Decisão	Implementar simuladores que expõem o contrato dos serviços reais, num ambiente separado, e acessá-los por configuração.
Estado	Aceita (mantida da entrega 1).
Consequências	Positiva: os serviços são escritos contra o contrato real e os simuladores podem provocar falhas para testar resiliência. Negativa: não reproduzem latência, indisponibilidade e regras reais do Banco Central.

ADR-09: Um servidor PostgreSQL com um banco por serviço

Seção	Conteúdo
Contexto	Seis servidores gerenciados estourariam o orçamento acadêmico (RT-7), mas Database per Service exige que um serviço não acesse os dados de outro.
Decisão	Um Flexible Server com seis bancos, cada um com um usuário que só tem permissão no próprio banco.
Estado	Aceita. No incremento atual, o banco ainda é H2 em memória (R-1).
Consequências	Positiva: isolamento lógico completo a um sexto do custo. Negativa: os bancos dividem CPU e memória; separar servidores fica como evolução se um serviço passar a dominar a carga.

10. Requisitos de Qualidade

Os objetivos de maior prioridade foram apresentados na seção 1.2. Aqui eles são organizados em árvore e desdobrados em cenários verificáveis, no formato do método ATAM apresentado em sala.

10.1 Árvore de Qualidade

Categoria (ISO 25010)	Qualidade	Prioridade	Cenário
Segurança	Autenticidade da movimentação	Alta	SC1, SC2
Segurança	Confidencialidade dos dados pessoais	Alta	SC3
Confiabilidade	Integridade do saldo	Alta	SC4, SC5
Confiabilidade	Ausência de duplicidade entre serviços	Alta	SC6
Confiabilidade	Isolamento de falhas	Média	SC7
Manutenibilidade	Modularidade e independência de deploy	Média	SC8, SC9
Eficiência / escalabilidade	Escala elástica por serviço	Média	SC10
Desempenho	Tempo de resposta do saldo	Média	SC11
Usabilidade	Facilidade de aprendizagem	Média	SC12

10.2 Cenários de Qualidade

Os cenários marcados com **testado** já são verificados por testes automatizados no código do incremento.

Id	Cenário	Resposta esperada	Como a arquitetura responde
SC1	Uma requisição chega sem token ou com token adulterado	Recusada com 401 antes de qualquer caso de uso	Gateway e filtro JWT de cada serviço (filtro JWT **testado**)
SC2	Um usuário autenticado tenta sacar ou transferir da carteira de outra pessoa	Recusada com 403; saldo inalterado	`ensureOwnedBy` em todo caso de uso **testado**
SC3	Um operador consulta o dossiê de um cliente	Acesso registrado com o id do operador; documentos só por link temporário	Auditoria no Serviço de Clientes e Blob privado
SC4	Uma transferência é recusada por saldo insuficiente	422, e nenhuma das duas carteiras nem o extrato mudam	Regras no agregado antes de qualquer gravação **testado**
SC5	Duas transferências simultâneas gastam o mesmo saldo	Uma é aplicada; a outra recebe 409; o saldo nunca fica negativo	Bloqueio otimista e transação única (ADR-07) **testado**
SC6	O evento `PixRecebido` é entregue duas vezes	O valor é creditado uma única vez	Consumidor idempotente pelo e2eId (seção 8.3)
SC7	O Serviço de Mensagens fica fora do ar por 30 minutos	PIX, saldo e extrato continuam funcionando; notificações são enviadas quando ele volta	Notificação assíncrona; eventos retidos no Service Bus
SC8	Um integrante adiciona uma nova fatia à Carteira	Nenhum teste das fatias existentes quebra	Fatias independentes em volta do domínio
SC9	O MFE Onboarding é publicado com uma correção	Shell e MFE Atendimento não são republicados	Module Federation e deploy separado
SC10	O número de PIX por minuto decuplica em uma data comemorativa	Só Pagamentos, Carteira e Funções ganham réplicas; os demais seguem iguais	Escala por serviço no Container Apps e nas Functions
SC11	O cliente abre a tela inicial com 1.000 lançamentos na carteira	Saldo exibido em menos de 1 segundo em rede móvel	Saldo armazenado na carteira, sem somar lançamentos
SC12	Uma pessoa que nunca usou o aplicativo precisa enviar um PIX por chave	Conclui sem ajuda em menos de 3 minutos	BFF entrega cada tela pronta; fluxo curto no app

11. Riscos e Débitos Técnicos

Riscos técnicos e dívidas assumidas conscientemente, ordenados por prioridade. Nenhum requisito funcional foi retirado do escopo: os itens tratam de limitações de ambiente, de decisões adiadas e da diferença entre o que está implementado e o que está projetado.

#	Risco ou débito	Impacto	Prioridade	Como será tratado
R-1	O código implementa 2 dos 6 serviços, e os dois ainda sobem como um único processo Spring Boot com um único banco (H2 em memória)	Alto	Alta	Separar `users` e `wallets` em dois deployables com bancos PostgreSQL próprios; como os módulos não se importam, a mudança é de empacotamento
R-2	Eventos e outbox ainda não implementados (componente tracejado da Figura 6)	Alto	Alta	Primeiro item da próxima entrega, começando por `MovimentacaoRegistrada` na Carteira
R-3	Consistência eventual entre serviços pode deixar estados intermediários visíveis	Médio	Alta	Sagas com compensação, consumidores idempotentes e telas que mostram operações pendentes
R-4	JWT assinado com HS256 e segredo compartilhado: todo serviço que valida o token também poderia emiti-lo	Médio	Média	Migrar para RS256, com chave privada só na Identidade e chave pública publicada (JWKS)
R-5	Mais peças para operar com uma equipe de 4 pessoas	Médio	Média	Serviços gerenciados, um pipeline padrão por serviço e observabilidade desde o início
R-6	Integração real com SPI/DICT nunca exercitada	Alto	Média	Manter o simulador fiel ao manual do PIX e provocar falhas nos testes
R-7	Partida a frio por causa da escala a zero	Baixo	Média	Réplica mínima 1 nos serviços do caminho do PIX se o cenário SC11 falhar
R-8	Leitura e gravação da carteira acontecem em transações separadas; conflitos viram 409 sem nova tentativa automática	Baixo	Baixa	Nova tentativa automática no BFF para operações idempotentes
R-9	Extrato sem paginação e ordenado só pelo instante do lançamento	Baixo	Baixa	Paginação por cursor e desempate pelo id
R-10	Conformidade regulatória real (autorização do Banco Central, prevenção à lavagem de dinheiro) fora do escopo	Médio	Baixa	Documentado como limite explícito do projeto acadêmico

12. Glossário

Termo	Definição
ADR	Architecture Decision Record: registro de uma decisão arquitetural com contexto, decisão, estado e consequências
Agregado	Conjunto de objetos de domínio alterado como uma unidade, com uma raiz que protege as regras (aqui, `Wallet`)
API Gateway	Ponto único de entrada que valida, limita e roteia as requisições para os serviços internos
arc42	Template para documentação de arquitetura de software criado por Gernot Starke e Peter Hruschka, organizado em 12 seções
Arquitetura-alvo	A arquitetura completa a ser implementada, descrita neste documento; o incremento atual implementa parte dela
BFF	Backend for Frontend: backend dedicado a um canal (app ou portal), que entrega a API no formato das suas telas
Bloqueio otimista	Controle de concorrência que não trava a leitura, mas recusa a gravação se o dado mudou desde que foi lido (coluna de versão)
C4 Model	Abordagem de visualização de arquitetura criada por Simon Brown, com quatro níveis: contexto, contêiner, componente e código
Clean Architecture	Organização em que as regras de negócio ficam no centro, sem depender de framework, e as dependências apontam sempre para dentro
Contêiner (C4)	Aplicação ou armazenamento de dados que precisa estar em execução para o sistema funcionar; não se refere a Docker
Database per Service	Cada microsserviço é dono exclusivo do seu banco; outros serviços acessam esses dados só pela sua API ou pelos seus eventos
DICT	Diretório de Identificadores de Contas Transacionais: cadastro de chaves PIX mantido pelo Banco Central
e2eId	Identificador ponta a ponta de uma transação PIX, único em todo o arranjo
EDA	Arquitetura Orientada a Eventos: serviços se comunicam publicando e assinando fatos de negócio
H2	Banco relacional em memória usado no incremento atual e nos testes, substituído pelo PostgreSQL na arquitetura-alvo
Idempotência	Propriedade de uma operação que, repetida, produz o mesmo resultado sem efeito adicional
JWT	JSON Web Token: token assinado que carrega a identidade do usuário entre canal e serviços
KYC	Know Your Customer: identificação e validação do cliente na abertura da conta
LGPD	Lei Geral de Proteção de Dados Pessoais, Lei nº 13.709/2018
Managed Identity	Identidade gerenciada pela Azure que permite a um serviço acessar outro sem credencial gravada
MFA	Autenticação com mais de um fator, como senha somada à biometria
Microfrontend	Parte da interface publicada de forma independente e composta com outras em tempo de execução
Module Federation	Mecanismo do empacotador JavaScript que permite a um Shell carregar módulos publicados separadamente
Outbox	Tabela onde o evento é gravado na mesma transação da mudança de estado, para depois ser publicado no barramento
Porta e adaptador	Interface definida pelo domínio (porta) e sua implementação técnica na borda (adaptador)
Problem Details (RFC 9457)	Formato padrão de resposta de erro em APIs HTTP
Saga	Sequência de transações locais em serviços diferentes, com ações de compensação se um passo falhar
Serverless	Modelo em que o código roda sob demanda, disparado por eventos, sem servidor administrado pela equipe

Termo	Definição
Software Architecture Canvas	Quadro de uma página do arc42 que resume objetivos, solução e avaliação da arquitetura (também publicado como Architecture Inception Canvas)
SPI	Sistema de Pagamentos Instantâneos: infraestrutura do Banco Central que liquida as transferências PIX
svc	Abreviação de *service* usada nos nomes dos apps no Azure Container Apps (por exemplo, `svc-carteira`)
Vertical Slice	Organização do código por caso de uso: cada fatia reúne tudo o que aquele caso de uso precisa, da entrada à regra